
Proyecto de desarrollo de aplicaciones web NexoPostal

**CICLO FORMATIVO DE GRADO SUPERIOR
Desarrollo de Aplicaciones Web (IFCS03)**

Curso 2025-26

Autor/a/es:

Ángel Sánchez Gasanz

Tutor/a:

Rafael Manjavacas Lara

**Departamento de Informática y Comunicaciones
I.E.S. Luis Vives**

1 INTRODUCCION

...

1.1 OBJETIVO

El objetivo principal de este Trabajo Fin de Grado ha sido el análisis, diseño, desarrollo e implantación de una plataforma software completa para la gestión de servicios de paquetería. El proyecto recibe el nombre de NexoPostal y nace con la finalidad de reproducir, en un entorno académico, pero con criterios técnicos realistas, el funcionamiento de una empresa de transporte y distribución postal moderna. La idea central del sistema es cubrir de forma integral todo el ciclo de vida de un envío, desde el momento en que un cliente calcula el precio y realiza el pago hasta la entrega final del paquete por parte de un repartidor, pasando por los procesos internos de admisión, clasificación, movimientos logísticos, control operativo y seguimiento en tiempo real.

El trabajo no se ha limitado a crear una única aplicación web, sino que se ha planteado como una solución distribuida y modular, compuesta por varias interfaces de usuario y varios microservicios especializados. Este enfoque permite representar de manera más fiel la complejidad de un sistema empresarial real, donde las necesidades de un cliente final no son las mismas que las de un operario de logística o las de un repartidor de última milla. Por ello, NexoPostal se estructura en tres aplicaciones frontend diferenciadas, un gateway de acceso, varios microservicios backend, distintas bases de datos y diversas integraciones externas.

De manera más concreta, los objetivos específicos del proyecto han sido los siguientes:

- Diseñar una aplicación para clientes que permita registrarse, iniciar sesión, calcular tarifas, crear envíos, pagar online, consultar el estado de los paquetes y gestionar su perfil.
- Desarrollar una intranet operativa orientada a personal interno para gestionar la admisión de paquetes, la clasificación por CTAs, las asignaciones de tareas, el seguimiento interno y el escaneo logístico.
- Implementar una aplicación para repartidores enfocada a la gestión de rutas, visualización de entregas, confirmación de entregas, seguimiento GPS y escaneo de códigos.
- Construir una arquitectura backend basada en microservicios con separación de responsabilidades entre autenticación, ciudadano, logística y reparto.
- Incorporar mecanismos de comunicación en tiempo real para que el cliente pueda seguir su envío y para que los operarios reciban notificaciones operativas.
- Integrar un sistema de pago realista mediante Stripe Checkout, junto con la generación automática de factura y etiqueta en PDF.
- Coordinar la comunicación interna entre servicios para sincronizar estados, registrar hitos operativos y materializar el paso desde la clasificación interna hasta la última milla.
- Preparar el sistema para su despliegue en contenedores Docker, con pipeline de integración y despliegue continuo.

En definitiva, el objetivo no ha sido únicamente programar una aplicación funcional, sino demostrar la capacidad de analizar un problema empresarial amplio, proponer una arquitectura adecuada, implementar una solución mantenible y justificar técnicamente cada una de las decisiones adoptadas.

1.2 ALCANCE

El alcance del proyecto abarca el desarrollo de una plataforma completa de paquetería con cobertura funcional para tres perfiles de uso principales: cliente final, personal interno de operaciones y personal de reparto. Para responder a estas necesidades, el sistema se divide en distintos bloques funcionales y técnicos.

En primer lugar, el alcance incluye tres aplicaciones frontend desarrolladas con Angular:

- Una aplicación orientada al cliente final, accesible desde la web principal, donde el usuario puede registrarse, autenticarse, consultar tarifas, crear envíos, realizar pagos, hacer seguimiento y gestionar sus datos personales.
- Una intranet operativa, destinada a la gestión interna de centros de tratamiento, tareas logísticas, movimientos entre nodos, incidencias y escaneos.
- Una aplicación específica para repartidores, preparada para ejecutarse en dispositivos móviles, con soporte para visualizar rutas, registrar entregas, enviar ubicación y continuar trabajando incluso con conectividad limitada.

En segundo lugar, el alcance incluye la capa backend basada en ASP.NET Core y microservicios. Los módulos implementados son:

- Módulo de autenticación y gestión de usuarios.
- Módulo ciudadano, responsable del dominio de envíos, tarifas, pagos, perfil y tracking público.
- Módulo intranet o logística, centrado en CTAs, asignaciones, movimientos y operativa interna.
- Módulo de reparto, dedicado a repartidores, rutas, entregas y localización.
- API Gateway para centralizar el acceso desde los clientes y aplicar reglas de autorización sobre las rutas.

En tercer lugar, el alcance contempla la infraestructura de datos y despliegue:

- Una base de datos PostgreSQL separada para cada microservicio principal.
- Uso de Docker y Docker Compose tanto en desarrollo local como en producción.
- Nginx como proxy inverso y punto de entrada para los distintos dominios.
- publicación de imágenes en GHCR y despliegue automatizado en un VPS.

Además, el proyecto integra servicios y tecnologías externas que amplían su realismo y valor técnico:

- Stripe Checkout para pagos online.
- SignalR para eventos en tiempo real.
- SMTP para envío de correos de confirmación y recuperación de contraseña.
- Leaflet para la representación cartográfica de la ruta del repartidor.
- html5-qrcode para lectura de códigos mediante Cámara.

No obstante, también es importante delimitar lo que queda fuera del alcance actual del proyecto. Aunque la plataforma es plenamente funcional, no pretende cubrir todas las capacidades de una empresa de paquetería de gran escala. Por ejemplo, no se ha implementado una app móvil nativa, ni un motor propio de navegación giro a giro, ni una plataforma de observabilidad distribuida completa, ni un bus de eventos empresarial con patrones outbox/inbox ya consolidados. Tampoco se ha desarrollado un algoritmo avanzado de optimización de rutas basado en tráfico real, capacidad del vehículo o ventanas temporales de entrega. Estas mejoras se plantean como evolución natural del sistema y se recogen en el apartado de trabajo futuro.

1.3 JUSTIFICACIÓN

La justificación del proyecto se apoya en tres dimensiones: la necesidad funcional que resuelve, el valor formativo que aporta y la calidad técnica del reto afrontado.

Desde el punto de vista funcional, el sector de la paquetería y la logística necesita soluciones que ofrezcan trazabilidad, rapidez operativa, automatización de procesos y una experiencia digital coherente para todos los actores implicados. El cliente final demanda conocer el estado de su envío, pagar con comodidad y disponer de información clara. El personal de operaciones necesita herramientas para clasificar, asignar, mover y controlar los paquetes sin depender de procesos manuales dispersos. Los repartidores, por su parte, requieren acceso a rutas, entregas, evidencias y posicionamiento, incluso en situaciones de movilidad o conectividad irregular. NexoPostal responde a esta necesidad proponiendo una única plataforma modular que conecta todos estos puntos.

Desde el punto de vista académico, el proyecto tiene especial interés porque permite poner en práctica conocimientos de programación frontend y backend, bases de datos, arquitecturas distribuidas,

integración continua, seguridad, despliegue y documentación técnica. No se trata de una simple aplicación CRUD, sino de un sistema con varios dominios, distintos tipos de usuarios, reglas de negocio reales, comunicación entre servicios y procesos asíncronos. Esto convierte el TFG en una muestra mucho más completa de competencias profesionales.

también está justificado por el aprendizaje que aporta a nivel de arquitectura. En lugar de concentrar toda la lógica en una aplicación monolítica, se ha optado por separar la solución en componentes especializados. Esta decisión obliga a trabajar conceptos como autenticación con JWT, paso de contexto entre servicios, coordinación de eventos, seguridad interservicio, gateways, gestión de configuración y despliegue por contenedores. En otras palabras, el proyecto no solo resuelve una necesidad funcional, sino que reproduce problemas y patrones habituales en entornos profesionales.

Por último, NexoPostal está justificado por su potencial de evolución. La base construida permite seguir ampliando el sistema con nuevas capacidades, como notificaciones push reales, analítica operativa, inteligencia en la asignación de rutas, dashboards avanzados, almacenamiento centralizado de evidencias o integración con proveedores externos. Esto convierte al proyecto en una plataforma extensible y no en un ejercicio cerrado de corto recorrido.

2 IMPLEMENTACIÓN

2.1 ANÁLISIS DE LA APLICACIÓN

2.1.1 Contexto del problema

Antes de definir la solución, fue necesario analizar el problema de negocio que se quería resolver. En una empresa de paquetería conviven varios procesos que, aunque para el cliente parezcan una sola operación, en realidad implican diferentes sistemas y responsables: alta del envío, cálculo del precio, pago, generación de documentación, admisión en oficina o sistema, clasificación en centros logísticos, transporte troncal entre zonas, asignación a reparto, entrega final y consulta de trazabilidad. Si cualquiera de estos pasos falla o queda desconectado del resto, la experiencia del usuario se resiente y la eficiencia operativa disminuye.

En muchos entornos tradicionales, la información del cliente, la logística interna y el reparto final se manejan con herramientas poco integradas o con procesos manuales. Esto provoca problemas como falta de visibilidad del estado real del paquete, duplicidad de datos, demoras en la actualización de estados, escasa automatización y baja capacidad de reacción ante incidencias. El análisis inicial del proyecto planteó que una solución adecuada debe ser capaz de unificar la visión del cliente y la visión interna, manteniendo a la vez una separación clara de responsabilidades entre los distintos módulos.

2.1.2 Actores y perfiles del sistema

El sistema identifica varios perfiles de usuario, cada uno con objetivos y permisos diferentes.

- Cliente: es el usuario final que crea envíos, paga, consulta tarifas, hace seguimiento del pedido y gestiona su cuenta.
- Administrador: perfil interno con capacidad de gestión global del sistema y acceso ampliado a la intranet.
- Operario de oficina: usuario vinculado al tratamiento de paquetes en oficina, especialmente en procesos de recepción y entrega en punto físico.
- Operario CTA: perfil orientado a los CTAs y a tareas de clasificación, asignaciones y movimientos logísticos.
- Supervisor: perfil con funciones de coordinación y mayor capacidad de gestión sobre tareas internas y de equipo.

- Repartidor: usuario de última milla encargado de ejecutar rutas, registrar entregas, reportar incidencias y enviar ubicación.
- Jefe de reparto: perfil responsable de la planificación y supervisión de la última milla.

La existencia de varios actores con necesidades distintas justifica la separación de interfaces y servicios. No tiene sentido que un cliente vea los mismos estados o acciones que un repartidor, del mismo modo que un operario de CTA necesita datos internos y no solo información pública de tracking.

2.1.3 Requisitos funcionales

Del análisis del problema se derivaron los principales requisitos funcionales del sistema.

En el área de cliente, la plataforma debe permitir:

- Registro de nuevos usuarios.
- Inicio de sesión y recuperación de contraseña.
- Consulta pública de tarifas.
- Creación de envíos con datos completos de remitente, destinatario y paquete.
- Selección de oficina de admisión y elección de entrega final en domicilio u oficina, según el caso.
- Pago online mediante una pasarela externa.
- Generación de etiqueta y factura.
- Seguimiento público por número de envío.
- Gestión del perfil y de las direcciones favoritas.
- Consulta del historial de envíos del usuario.

En el área interna de logística, la solución debía ofrecer:

- Resolución automática del CTA de destino a partir del código postal.
- Admisión de paquetes en la red interna.
- Creación automática de movimientos troncales cuando el origen y el destino corresponden a nodos distintos.
- Notificaciones en tiempo real a operarios de los CTAs.
- Creación y seguimiento de asignaciones de tareas.
- Registro de incidencias.
- Seguimiento interno del paquete por número de expedición o seguimiento.
- Cambios de estado interno por parte de operarios y repartidores.
- Escaneo individual y por lotes de códigos internos.

En el área de reparto, el sistema debía incluir:

- Gestión del perfil del repartidor.
- Consulta de la ruta del día y sus entregas.
- Inicio y finalización de la ruta.
- Confirmación de entrega con distintos resultados posibles.
- Registro de evidencias como nombre del receptor, DNI, firma o foto.
- Envío periódico de ubicación GPS.
- Persistencia temporal de reintentos automáticos.
- Escaneo de expediciones desde la propia app de reparto.

2.1.4 Requisitos no funcionales

Además de los requisitos funcionales, el proyecto debía cumplir una serie de condiciones técnicas que garantizaran su calidad.

- Modularidad: separar dominios para reducir el acoplamiento y facilitar mantenimiento.
- Escalabilidad: permitir que los componentes evolucionen de forma relativamente independiente.
- Seguridad: usar autenticación JWT, control de roles y protección básica entre microservicios.
- Trazabilidad: registrar y exponer estados públicos e internos de forma coherente.
- Usabilidad: interfaces diferenciadas y adaptadas a cada tipo de usuario.
- Portabilidad: facilitar ejecución local y despliegue remoto mediante contenedores.

- Resiliencia: soportar reconexiones en tiempo real y cierta tolerancia a conectividad limitada en reparto.
- Mantenibilidad: organizar el código por capas, servicios, DTOs, repositorios y modelos.

Estos requisitos no funcionales no se definieron como una lista teórica separada del desarrollo, sino que condicionaron directamente la forma de construir el sistema. La modularidad explica la división en microservicios y en tres frontends distintos; la portabilidad justifica el uso de Docker y Docker Compose; la seguridad se materializa en JWT, roles y claves internas entre servicios; la resiliencia se refleja en la reconexión automática de SignalR; y la mantenibilidad se consigue gracias a una estructura separada por controladores, servicios, repositorios, DTOs y modelos. En otras palabras, buena parte de la arquitectura técnica de NexoPostal es una consecuencia directa de estos requisitos no funcionales.

2.1.5 Reglas de negocio clave

Más allá de los requisitos generales, durante el análisis fue necesario identificar un conjunto de reglas de negocio concretas que determinan el comportamiento real de la plataforma. Estas reglas son especialmente importantes porque convierten el sistema en una solución coherente y no en una simple suma de pantallas y endpoints.

- El cálculo de tarifas se resuelve siempre en backend. La web no utiliza un simulador local distinto, sino que consulta un motor único de tarifas. Esto garantiza consistencia entre el precio estimado, el precio pagado y el coste finalmente asociado al envío.
- El peso facturable del paquete no depende solo del peso real. El sistema compara el peso real con el peso volumétrico, calculado a partir de las dimensiones, y utiliza el mayor de ambos para la tarificación.
- Existen restricciones físicas y comerciales sobre el paquete. El formulario y la lógica de negocio limitan el peso máximo a 30 kg, imponen dimensiones mínimas para poder etiquetar el bulto y aplican recargo cuando la suma de dimensiones supera 210 cm o el lado mayor excede 170 cm.
- La operativa contempla casuísticas territoriales reales. Si el código postal de origen o destino corresponde a Canarias, el sistema exige la identificación fiscal del remitente o del destinatario según el caso, introduciendo una regla especial que no afecta al resto del territorio.
- En la contratación online, el origen se articula mediante una oficina de admisión seleccionada por el remitente, mientras que la entrega final puede resolverse en domicilio u oficina. Esta distinción afecta a la experiencia de usuario, a la construcción de direcciones y a la información asociada a la expedición.
- Cada envío dispone de dos identificadores distintos. El número de seguimiento público se utiliza en la web de clientes y en el tracking externo, mientras que el número de expedición interno se utiliza en intranet, escaneo y reparto. Esta dualidad separa la visibilidad pública de la operativa interna.
- El envío no entra en la red logística por el mero hecho de rellenar el formulario. Primero se crea en estado pendiente de pago; solo cuando el pago se confirma se marcan estados operativos, se generan documentos y se notifica la admisión a logística.
- El estado público del envío no es una variable independiente y arbitraria, sino una proyección simplificada del estado interno. Esto permite que el cliente vea una trazabilidad comprensible sin exponer toda la complejidad operativa del circuito logístico.
- La admisión del paquete resuelve automáticamente el CTA de destino a partir del código postal. Si origen y destino pertenecen a nodos distintos, se crea además un movimiento troncal con el tipo de transporte adecuado.
- La última milla no nace por autoasignación directa desde la admisión. El punto de corte operativo es el estado DisponibleParaReparto, que incorpora el paquete a la bandeja persistente de reparto para que el JefeReparto lo planifique manualmente.
- El seguimiento GPS del repartidor no debe estar activo en cualquier contexto. La app móvil solo intensifica el tracking cuando la ruta está realmente en curso, evitando gasto innecesario de batería y eventos irrelevantes.

- La falta de conectividad puntual se gestiona mediante reintentos automáticos en el GPS del repartidor, con backoff exponencial ante errores consecutivos.

Estas reglas son fundamentales para entender el valor del proyecto. No se limitan a ser detalles técnicos de implementación, sino que explican por qué el comportamiento observado en cliente, intranet y reparto es coherente entre sí.

2.1.6 Procesos de negocio principales

Una vez identificados los requisitos, se definieron los procesos de negocio que articulan la plataforma.

Proceso 1. Alta y autenticación de cliente

El usuario se registra en la aplicación de clientes mediante email, contraseña y nombre completo. Estos datos se almacenan en el microservicio de autenticación, que emite el token JWT necesario para el resto de las operaciones privadas. Una vez autenticado, el cliente puede acceder a su panel, consultar sus envíos, mantener su agenda de direcciones y operar sobre el resto de los servicios de forma segura. Este proceso es transversal, ya que actúa como puerta de entrada al resto de funcionalidades de valor.

Proceso 2. Cotización y creación del envío

El cliente introduce peso, dimensiones, origen y destino del paquete. A partir de esos datos, el backend calcula el precio considerando zona, peso real, peso volumétrico, posibles recargos y tipo de servicio. Una vez completados remitente y destinatario, el sistema no crea directamente un envío pagado, sino que prepara un envío pendiente de pago y genera la sesión de Stripe Checkout asociada. Este matiz es importante porque separa la intención de compra de la confirmación real del cobro.

Proceso 3. Pago y admisión logística

Cuando Stripe devuelve un resultado satisfactorio, el sistema verifica la sesión y actualiza el envío. En ese momento se marca como pagado, se registran la fecha de pago y los estados iniciales operativos, se genera la etiqueta PDF, se genera la factura PDF y se envían ambos documentos por correo electrónico. A continuación, el microservicio ciudadano notifica a logística para que el paquete entre en la red interna. En caso de cancelación o fallo, el envío puede permanecer pendiente de pago y volver a intentarse sin perder el contexto de la expedición.

Proceso 4. Clasificación y movimientos internos

El módulo de intranet analiza el código postal de destino y determina el CTA que debe gestionar el paquete. Si el código postal de origen remite a un nodo diferente, se crea automáticamente un movimiento troncal entre ambos centros. El sistema decide además el tipo de transporte en función del contexto y de la urgencia del envío. Una vez registrado el paquete en la red, se envían notificaciones en tiempo real a los operarios logísticos del CTA correspondiente para iniciar la clasificación y las tareas asociadas.

Proceso 5. Seguimiento interno del paquete

La operativa interna trabaja principalmente con el número de expedición. Este identificador se usa para escaneo, seguimiento detallado, asignaciones de tareas y actualización de estados. A diferencia del tracking público, el seguimiento interno refleja pasos más finos del circuito logístico, como clasificación, movimientos entre centros, intentos fallidos de entrega, incidencias específicas o devoluciones. Gracias a ello, la plataforma distingue entre lo que necesita ver un cliente y lo que necesita gestionar un operario.

Proceso 6. Liberación a reparto y planificación de última milla

Cuando el circuito interno alcanza el estado DisponibleParaReparto, la intranet registra el paquete en la bandeja persistente del microservicio de reparto mediante un endpoint interno protegido. A partir de ese momento, el JefeReparto visualiza los pendientes de su ámbito, decide si crea una ruta nueva o si incorpora el paquete a una ruta existente y asigna la parada correspondiente. Este modelo desacopla la clasificación de la planificación de calle y refleja con mayor fidelidad la operativa real.

Proceso 7. Ejecución de ruta y entrega

El repartidor inicia sesión en su aplicación y consulta la ruta asignada del día. Cuando comienza la jornada, marca la ruta como iniciada y la app activa el seguimiento GPS, el mapa operativo y la sincronización de eventos. Durante la ruta, el repartidor puede seleccionar entregas, navegar hacia la siguiente parada, registrar el resultado de cada intento y adjuntar evidencias como receptor, DNI, firma, foto u observaciones. Al finalizar, cierra la ruta con observaciones de jornada y detiene el seguimiento activo.

Proceso 8. Tracking público en tiempo real

Mientras el paquete avanza por la red, el módulo ciudadano proyecta la operativa interna en una traza pública comprensible. El cliente consulta inicialmente el estado por HTTP y, a continuación, queda suscrito mediante SignalR a futuras actualizaciones del número de seguimiento. Cuando reparto informa de una nueva ubicación o de un evento de entrega, ciudadano transforma esa información en cambios de estado, incidencias o confirmaciones visibles para el cliente. De este modo, la trazabilidad pública se alimenta de procesos operativos reales y no de mensajes simulados.

Conviene destacar que estos procesos no funcionan de manera aislada. El valor del sistema reside precisamente en que un proceso alimenta al siguiente: la autenticación permite operar; la cotización conduce al pago; el pago dispara la admisión; la admisión genera movimiento y tareas internas; la clasificación libera el paquete a reparto; el reparto actualiza el tracking; y el tracking devuelve información al cliente en tiempo real. Esta continuidad operativa es una de las claves de NexoPostal como proyecto.

2.1.7 Entidades principales del dominio

Del análisis funcional surgieron las entidades que estructuran el sistema.

En el dominio de autenticación destaca la entidad `ApplicationUser`, que representa al usuario del sistema. Esta entidad no solo almacena las credenciales básicas, sino también el nombre completo, el código de empleado cuando procede, la fecha de registro y el rol asignado. Gracias a ella, el mismo sistema de autenticación puede dar soporte tanto a clientes como a perfiles internos.

En el dominio ciudadano destacan:

- `ClientePerfil`, con datos ampliados del cliente y dirección predeterminada.
- `DireccionFavorita`, para almacenar agenda personal de direcciones.
- `Envío`, entidad central del dominio, con doble identificador: número de seguimiento público y número de expedición interno.

La entidad `ClientePerfil` amplía la información del usuario autenticado con datos pensados para la operativa de la aplicación de clientes, como el teléfono, el documento identificativo o la dirección predeterminada. La entidad `DireccionFavorita` permite persistir una agenda reutilizable que simplifica la contratación de nuevos envíos. La entidad `Envío`, por su parte, concentra la mayor parte del valor de negocio: origen, destino, datos de remitente y destinatario, códigos postales, peso, dimensiones, coste, tarifa, estado público, estado interno, estado de pago y referencias a documentos y sesiones de Stripe.

En el dominio logístico destacan:

- `CentroTratamiento`, que modela los CTAs de la red.
- `RutaCta`, para definir relaciones entre zonas y centros.
- `OperarioCta` y `OperarioOficina`, para vincular personal interno a nodos operativos.
- `AsignacionPaquete`, que representa una tarea concreta sobre un envío dentro de un CTA.
- `MovimientoPaquete`, que representa un desplazamiento troncal entre dos centros.
- `Incidencia`, para registrar excepciones del proceso.

`CentroTratamiento` actúa como nodo logístico principal y se complementa con reglas de clasificación por prefijos de código postal. `OperarioCta` y `OperarioOficina` permiten modelar quien trabaja en cada ámbito y con que rol. `AsignacionPaquete` representa trabajo físico concreto sobre un bulto, con responsable, creador, estado, urgencia y fechas operativas. `MovimientoPaquete` modela el trayecto troncal entre dos CTAs, incluyendo origen, destino, estado y tipo de transporte. Finalmente, `Incidencia` recoge las

excepciones del flujo para que el sistema no pierda trazabilidad cuando algo se desvía de la operativa normal.

En el dominio de reparto destacan:

- Repartidor, perfil especializado asociado a una oficina de referencia.
- RutaReparto, agrupación de entregas para una jornada y un repartidor concretos.
- EntregaPaquete, unidad operativa de última milla sobre un paquete de una ruta.

La entidad Repartidor conecta la identidad del usuario con su rol de última milla, su oficina de referencia y su contexto operativo. RutaReparto representa la unidad de trabajo diaria del repartidor y contiene información sobre fecha, estado, horario y agrupación de entregas. EntregaPaquete es la entidad que más se acerca a la realidad de la última milla, ya que registra dirección, destinatario, orden de parada, estado, intento, evidencias y coordenadas de entrega.

La decisión de utilizar dos identificadores distintos para un envío es especialmente relevante. El número de seguimiento público se expone al cliente y se usa para el tracking web. El número de expedición interno se reserva a la operativa de intranet y reparto. Esta separación reduce exposición innecesaria de información interna y permite trabajar con dos niveles de granularidad funcional.

También resulta importante la distribución de la propiedad de los datos. Auth es dueño de la identidad y de las credenciales; Ciudadano es dueño de la contratación, del perfil de cliente, del pago y del tracking público; Intranet es dueña de la gestión de CTAs, tareas y movimientos internos; y Reparto es dueño de rutas, entregas y posición del repartidor. Esta separación de responsabilidades es una de las claves arquitectónicas del proyecto y conviene mantenerla visible desde el propio análisis funcional.

2.2 DISEÑO

2.2.1 Diseño arquitectónico general

La arquitectura de NexoPostal se ha diseñado siguiendo un enfoque de microservicios con separación clara de dominios. Esta decisión no se adoptó por moda tecnológica, sino porque encaja con el problema a resolver. La autenticación, la contratación de envíos, la operativa logística interna y el reparto de última milla tienen reglas de negocio diferentes, ritmos de cambio distintos y necesidades de seguridad particulares. Si toda la lógica se hubiese concentrado en un único backend monolítico, el resultado habría sido una base de código más acoplada, menos legible y más difícil de justificar académica y técnicamente.

Desde el punto de vista lógico, la solución puede dividirse en cuatro planos:

- Plano de presentación, compuesto por tres clientes Angular independientes: clientes, intranet y driver app.
- Plano de acceso, compuesto por Nginx y el API Gateway, que concentran entrada, enrutado, dominios y políticas de autorización.
- Plano de negocio, formado por los microservicios Auth, Ciudadano, Intranet y Reparto.
- Plano de persistencia e integración, formado por PostgreSQL por servicio, archivos JSON de apoyo, correo SMTP, Stripe y SignalR.

La responsabilidad de cada bloque es la siguiente:

- Nginx actúa como puerta de entrada externa y publica las tres aplicaciones bajo dominios diferenciados.
- El API Gateway centraliza el acceso HTTP desde los clientes y decide que rutas son públicas y cuales requieren autenticación.
- El microservicio Auth gestiona identidades, login, registro, refresh token, recuperación de contraseña y datos base del usuario.
- El microservicio Ciudadano concentra la lógica visible para el cliente: envíos, tarifas, pagos, documentos, perfil, direcciones y tracking público.
- El microservicio Intranet modela la red logística interna: CTAs, operarios, asignaciones, movimientos, incidencias, admisión y notificaciones operativas.
- El microservicio Reparto se ocupa de la última milla: repartidores, rutas, entregas, posición del repartidor y sincronización de eventos de entrega.

Esta arquitectura presenta varias ventajas. En primer lugar, cada módulo concentra la lógica de su dominio y puede evolucionar con menor impacto sobre el resto. En segundo lugar, permite una división más limpia de tablas, endpoints y reglas de negocio. En tercer lugar, facilita el despliegue por contenedores y el aislamiento de fallos. Finalmente, tiene gran valor académico porque obliga a resolver integración, seguridad, orquestación y consistencia de datos entre módulos, cuestiones mucho más cercanas a un sistema real que a una aplicación CRUD convencional.

2.2.2 Diseño de la arquitectura física

La arquitectura física del proyecto se apoya en contenedores Docker tanto en local como en producción. Esto permite que el entorno de desarrollo y el entorno desplegado compartan una filosofía común de ejecución, algo muy valioso para reducir diferencias de comportamiento y simplificar las pruebas.

En desarrollo local, la plataforma se levanta con `docker-compose.yml` mediante `docker compose up -d --build`. Todos los servicios se conectan a una red común y el proxy Nginx expone la web de clientes en el puerto 80, la intranet en el puerto 4202 y la app de reparto en el puerto 4201. Dentro de esa misma red conviven el gateway, los cuatro microservicios principales y las cuatro bases de datos PostgreSQL. Además, las bases de datos exponen puertos locales separados para facilitar depuración y acceso en desarrollo. Este modelo permite reproducir el comportamiento completo de la plataforma sin instalaciones manuales adicionales de bases de datos o backends.

En producción, el despliegue se realiza con `docker-compose.production.yml`, pero en lugar de construir contenedores en el propio servidor se consumen imágenes previamente publicadas en GHCR. Esta decisión reduce el trabajo del VPS, acorta el tiempo de despliegue y separa claramente la fase de compilación de la fase de ejecución. En este escenario solo se exponen externamente los puertos 80 y 443, mientras que las bases de datos quedan aisladas en la red interna. Esta organización es más segura y cercana a una operativa profesional.

Otro elemento importante de la arquitectura física es la diferenciación por dominios. Nginx sirve la web de clientes bajo el dominio principal, la intranet bajo un subdominio propio y la app de reparto bajo otro distinto. Además, el proxy está configurado para manejar correctamente las conexiones WebSocket necesarias para SignalR, tanto en el tracking público como en las notificaciones internas de la intranet. Esto demuestra que el sistema no solo está diseñado a nivel lógico, sino también preparado para funcionar como un conjunto de aplicaciones publicadas de forma coherente.

La configuración por variables de entorno también forma parte del diseño físico. Los contenedores consumen archivos `.env` para resolver cadenas de conexión, claves JWT, configuración de Stripe, credenciales SMTP y URLs internas de los servicios. Gracias a ello, el mismo código puede ejecutarse en distintos entornos sin necesidad de modificar ficheros fuente.

2.2.3 Diseño de datos

El diseño de datos sigue el principio de base de datos por servicio. Cada microservicio mantiene su propio esquema y sus propias tablas, evitando dependencias fuertes entre bases de datos y reduciendo el acoplamiento. Esta estrategia obliga a pensar cuidadosamente que módulo es dueño de cada dato y como se relacionan las piezas del sistema sin compartir directamente tablas entre dominios.

La distribución principal de la persistencia es la siguiente:

- Auth mantiene las tablas de identidad, usuarios, credenciales y tokens asociados a la sesión.
- Ciudadano mantiene perfiles de cliente, direcciones favoritas y envíos contratados, incluyendo sus estados, coste, datos del remitente y del destinatario, y referencias de pago.
- Intranet mantiene CTAs, rutas de clasificación, relaciones con operarios, asignaciones, movimientos e incidencias.
- Reparto mantiene repartidores, rutas diarias y entregas de última milla.

Para enlazar información entre módulos se utilizan identificadores compartidos o referencias cruzadas de tipo lógico, no relaciones físicas entre motores distintos. Algunos ejemplos especialmente relevantes son los siguientes:

- IdentityUserId, que conecta los datos de Auth con perfiles en Ciudadano y perfiles de repartidor en Reparto.
- NumeroSeguimiento, que identifica públicamente un envío y se utiliza en tracking y comunicación con el cliente.
- NumeroExpedicion, que sirve de referencia operativa transversal entre Ciudadano, Intranet y Reparto.
- Identificadores internos como Rutald, Entregald o Ctald, que organizan la operativa propia de cada dominio.

Este diseño obliga a asumir una consecuencia importante: la consistencia entre servicios no se obtiene mediante relaciones SQL directas, sino mediante contratos de integración y actualizaciones coordinadas. Precisamente por eso la memoria debe explicar no solo las entidades aisladas, sino también como se sincronizan los estados entre módulos.

El tratamiento del catálogo de oficinas ha evolucionado respecto a versiones anteriores del proyecto. En la implementación actual, las oficinas no se consumen únicamente desde un JSON estático cargado en memoria, sino que existen modelos persistidos y APIs específicas para listarlas, buscarlas y administrarlas. Ciudadano mantiene un directorio de oficinas consultable por backend y cacheable en frontend para mejorar la experiencia de búsqueda. Intranet, por su parte, mantiene el maestro operativo de oficinas postales y permite altas, ediciones, activaciones y desactivaciones desde paneles administrativos. Siguen existiendo seeders y servicios de apoyo basados en catálogos estructurados para inicializar o resolver oficinas, pero el estado que consumen las aplicaciones ya no depende exclusivamente de un fichero local del frontend. Esto mejora la trazabilidad, facilita el mantenimiento y evita tener que redistribuir los clientes web ante cambios operativos del catálogo.

También existen mecanismos de persistencia ligera en cliente. Por ejemplo, la autenticación en frontend utiliza almacenamiento local para conservar token y contexto de usuario.

2.2.4 Diseño de seguridad

La seguridad se ha diseñado en varias capas, combinando autenticación, autorización, separación por roles, aislamiento de entornos y protección básica entre servicios internos.

A nivel de usuario final, el acceso a rutas protegidas se gestiona mediante JWT. El frontend conserva el token y lo envía en las peticiones autenticadas, mientras que los microservicios validan firma, emisor, audiencia y expiración. Un detalle relevante del proyecto es que la validación se configura sin margen de cinco minutos, utilizando `ClockSkew = TimeSpan.Zero`, lo que endurece el comportamiento temporal de la sesión y evita aceptar tokens expirados durante un intervalo extra.

A nivel de entrada al sistema, el gateway añade una segunda capa de control porque distingue entre rutas públicas y privadas. Login, registro, refresh, reseteo de contraseña, tracking público, cálculo de tarifas, búsqueda de oficinas y webhook de Stripe son rutas abiertas por necesidad funcional. El resto exige contexto autenticado. Este punto es importante porque la frontera de acceso no depende solo del frontend, sino también del backend central de entrada.

A nivel de roles, cada aplicación cliente restringe lo que puede hacer el usuario según su perfil. La web de clientes admite únicamente usuarios con rol Cliente. La intranet presenta opciones diferenciadas para cuatro perfiles: Admin, OperarioOficina (atiende ventanilla, da altas presenciales y escanea en oficinas postales), OperarioCTA (trabaja en la nave de clasificación y gestiona movimientos troncales) y Supervisor (coordina el CTA, crea o reasigna tareas y supervisa incidencias y métricas). La app de reparto distingue entre Repartidor (ejecuta la ruta y confirma entregas) y JefeReparto (planifica rutas, gestiona la bandeja de paquetes listos para reparto, redistribuye paradas y administra su equipo). Esta separación evita mezclar experiencias de usuario y refuerza la idea de aplicación especializada por contexto operativo.

Un endurecimiento importante introducido en la versión actual es la validación del estado de sesión desde el gateway contra un endpoint interno de Auth. Ya no basta con que el JWT sea formalmente válido: si una cuenta ha sido bloqueada por un administrador, el gateway puede detectar esa situación y cortar el acceso de forma inmediata. Esto reduce la ventana de riesgo en la que un usuario bloqueado podría seguir operando hasta la expiración natural del token.

La seguridad del dominio de autenticación también se refuerza con funcionalidades adicionales, como refresh tokens reales con rotación, revocación y recuperación de contraseña por correo. Estas capacidades mejoran la experiencia de usuario, pero también elevan el nivel del proyecto frente a un modelo mínimo de login con token simple.

En la comunicación interservicio, determinados endpoints internos se protegen mediante una X-Service-Key. Esta clave se usa, por ejemplo, para admitir paquetes desde Ciudadano hacia Intranet o para publicar tracking desde Reparto hacia Ciudadano. En varios puntos del sistema la comparación se realiza en tiempo constante, reduciendo problemas asociados a comparaciones inseguras de cadenas. Aunque este enfoque es suficiente para un MVP académico y para una arquitectura pequeña, la propia memoria debe dejar claro que en un escenario empresarial de mayor madurez sería recomendable evolucionar hacia esquemas más robustos, como JWT de servicio a servicio o mTLS.

Finalmente, la seguridad se extiende también a la capa de publicación. En producción se utiliza HTTPS con certificados en Nginx y se aplican cabeceras de seguridad propias de un entorno web real, como HSTS y protección frente a mezcla de contenido. Esto refuerza el sistema no solo desde el código, sino también desde la infraestructura de entrega.

2.2.5 Diseño de la comunicación entre servicios y de los eventos en tiempo real

Uno de los aspectos más interesantes del diseño de NexoPostal es que la integración entre módulos no se basa en una única técnica, sino en una combinación de llamadas HTTP entre microservicios y eventos push mediante SignalR. Este enfoque híbrido permite separar adecuadamente la operativa sin perder continuidad de negocio.

La comunicación síncrona entre servicios se utiliza cuando un módulo necesita provocar de forma inmediata una acción concreta en otro. Algunos ejemplos clave en la versión actual son los siguientes:

- Ciudadano notifica a Intranet que un envío pagado debe ser admitido en la red logística.
- Intranet registra en Reparto un paquete pendiente cuando el flujo de escaneo lo sitúa en el estado DisponibleParaReparto.
- Reparto comunica a Ciudadano ubicaciones, intentos de entrega y cierres de reparto para consolidar el tracking público.

Este modelo tiene la ventaja de ser sencillo de seguir y muy apropiado para un TFG, porque deja visibles las dependencias funcionales entre dominios. Sin embargo, también introduce una limitación conocida: al depender de peticiones directas, la robustez frente a fallos temporales de red es menor que en un sistema basado en mensajería duradera. Precisamente por eso el proyecto recoge como trabajo futuro la evolución hacia patrones como outbox e inbox.

En paralelo, el sistema utiliza SignalR en tres ámbitos bien diferenciados. El primero es el tracking público del cliente, donde el módulo Ciudadano expone un hub al que cualquier usuario puede suscribirse con un número de seguimiento. El segundo es la intranet, donde existe un hub interno autenticado que organiza a los usuarios por grupos de CTA y rol y distribuye eventos como tareas asignadas, movimientos recibidos o paquetes listos para reparto. El tercero es el módulo de reparto, que dispone de su propio hub para mantener sincronizadas las vistas operativas del equipo de última milla. Esta separación es muy importante porque el realtime público, el realtime logístico y el realtime de movilidad responden a necesidades distintas y no deben mezclarse.

Gracias a esta estrategia, el flujo completo puede describirse de forma coherente. Un cliente contrata y paga un envío; Ciudadano dispara la admisión interna; Intranet resuelve CTA, crea movimientos y encadena tareas operativas mediante escaneo; cuando la clasificación marca el paquete como DisponibleParaReparto, este se registra en la bandeja persistente del JefeReparto; a continuación, el módulo de reparto crea o reutiliza rutas, ejecuta entregas y reporta ubicación; y, finalmente, Ciudadano proyecta esa operativa al tracking visible para el cliente. El sistema, por tanto, no solo integra módulos, sino que mantiene un hilo narrativo único del dato a través de todos ellos.

2.2.6 Diseño de las interfaces de usuario

Uno de los aspectos más importantes del diseño ha sido separar claramente las interfaces según el tipo de usuario. El sistema no utiliza una única web para todo, sino tres clientes especializados.

A) Aplicación de clientes

La aplicación de clientes se ha diseñado para ofrecer una experiencia cercana a la de una empresa de paquetería real. Sus pantallas principales son:

- Inicio: punto de acceso general a la plataforma y a las funcionalidades públicas.
- Registro y login: acceso del cliente a su espacio personal.
- Calculadora de tarifas: permite estimar precios antes de contratar.
- Nuevo envío: asistente de varias etapas para preparar el paquete y realizar el pago.
- Buscador de oficinas: localiza oficinas por código postal o texto libre.
- Tracking: muestra el estado del envío y sus actualizaciones en tiempo real.
- Panel de usuario: concentra perfil, direcciones favoritas y listado de envíos.
- Pago exitoso y pago cancelado: gestión del retorno tras Stripe Checkout.
- Recuperación de contraseña: flujo de reseteo por correo.

La pantalla de nuevo envío merece especial atención porque concentra una gran parte del valor funcional del proyecto. Su diseño sigue un modelo por pasos. Primero se recogen los datos del remitente y la oficina donde se entregará físicamente el paquete. Después, los del destinatario. Por último, se introducen las características físicas del paquete y se calculan las tarifas disponibles. En la versión actual, la recogida domiciliaria de origen no forma parte del flujo online: el remitente entrega siempre el paquete en una oficina postal, mientras que el destinatario puede recibirlo en domicilio o en oficina. El usuario puede reutilizar direcciones guardadas, buscar oficinas por ciudad o código postal y seleccionar la tarifa más adecuada. Esta estructura por pasos reduce la complejidad percibida y mejora la usabilidad.

Además, el formulario incorpora validaciones de negocio que acercan la aplicación a un caso real. Se comprueba el peso máximo, las dimensiones mínimas necesarias para etiquetado, la longitud máxima permitida y la aplicación de recargos por exceso de tamaño. También se exige DNI/NIF en envíos con Canarias, lo que introduce una casuística interesante en la experiencia de usuario.

B) Intranet operativa

La intranet tiene un enfoque completamente distinto. No prima la contratación o la simplicidad comercial, sino la operativa, la rapidez y la visibilidad del estado interno. Sus pantallas principales se organizan hoy por rol y por ámbito funcional:

- Login interno.
- Dashboard inicial con accesos distintos para OperarioOficina, OperarioCTA, Supervisor y Admin.
- Alta presencial en oficina, donde el operario registra un envío que el cliente entrega físicamente en ventanilla y cobra en efectivo o tarjeta.
- Gestión de CTA, donde supervisor y administración consultan métricas, tareas, movimientos e incidencias del centro.
- Asignaciones y escaneo integrado, donde se visualizan colas de trabajo, se crean tareas manuales, se reasignan o cancelan y se avanza cada expediente mediante cámara o entrada manual.
- Seguimiento interno, donde se busca un envío por número de expedición o seguimiento y se consulta su historial operativo.
- Gestión de equipo, centrada en operarios de CTA.
- BackOffice de administración, reservado a Admin, con módulos de usuarios, clientes, envíos, repartidores, CTAs, oficinas, tarifas, incidencias globales, movimientos globales y broadcast de notificaciones.

La vista de asignaciones con escaneo integrado es especialmente representativa del enfoque operativo del sistema. La misma pantalla sirve para procesar tareas propias, escanear paquetes, filtrar estados, ocultar completadas y, cuando el rol lo permite, crear o reasignar trabajo. Además, el proyecto incorpora casuísticas operativas reales, como el reporte de PaqueteFueraDeTareas, que permite registrar incidencias cuando un paquete aparece fuera de la cola esperada.

C) Aplicación de repartidores

La app de reparto se ha diseñado con orientación móvil y foco en la acción, pero en la versión actual ya no responde a un único tipo de usuario. Conviven dos experiencias claramente diferenciadas:

- Repartidor, con dashboard operativo, ruta activa, mapa, confirmación de entregas, GPS y escaneo puntual.
- JefeReparto, con dashboard propio, bandeja de paquetes disponibles para reparto, gestión de rutas del día, reasignación de paradas, mapa en tiempo real y gestión de su equipo de repartidores.

Esta separación es especialmente valiosa porque evita mezclar la experiencia de quien ejecuta entregas en calle con la de quien organiza la última milla desde la oficina. Ambas comparten identidad y parte del backend, pero no la misma interfaz ni nivel de decisión operativa.

La vista de ruta es la pantalla más importante del módulo de reparto. En ella se muestra la ruta asignada, el estado de la jornada, las paradas pendientes, las entregas completadas y las fallidas, así como un mapa con la posición del repartidor y los destinos disponibles. Desde esta misma pantalla el usuario puede iniciar o finalizar la ruta, centrar el mapa, abrir navegación externa y confirmar una entrega con toda la información asociada.

Este diseño responde a una realidad operativa clara: el repartidor no puede navegar entre múltiples pantallas complejas mientras conduce o realiza entregas. Por ello, se ha concentrado la mayor parte de la acción en una vista única y se ha dotado a esa vista de soporte GPS, reintentos y evidencias de entrega.

2.2.7 Manual de usuario resumido por procesos

Para reforzar el diseño funcional, puede describirse el uso de la plataforma mediante una secuencia de procesos.

Proceso de envío por parte del cliente:

1. El usuario accede a la web de NexoPostal y se autentica.
2. Accede a la opción de nuevo envío.
3. Introduce los datos del remitente, pudiendo seleccionar dirección u oficina.
4. Introduce los datos del destinatario, igualmente con posibilidad de dirección u oficina.
5. Informa peso y dimensiones del paquete.
6. El sistema calcula las tarifas disponibles y el usuario selecciona una.
7. Se crea la sesión de pago y el usuario es redirigido a Stripe.
8. Tras el pago, vuelve a la aplicación, donde se verifica la sesión y se confirma el envío.

Proceso de seguimiento por parte del cliente:

1. El usuario accede al apartado de tracking.
2. Introduce el número de seguimiento.
3. El sistema recupera por HTTP el estado actual y el historial disponible.
4. A continuación, el cliente queda suscrito por SignalR a las actualizaciones del envío.
5. Si el estado cambia mientras la vista está abierta, la información se actualiza en tiempo real.

Proceso de trabajo en intranet:

1. El operario inicia sesión en la intranet.
2. Consulta su CTA o conjunto de CTAs asignados.
3. Revisa métricas, notificaciones o tareas pendientes.
4. Crea o gestiona asignaciones sobre expediciones concretas.
5. Puede buscar envíos por número interno y cambiar estados según la operativa realizada.
6. En tareas más intensivas, utiliza el módulo de escaneo para registrar entradas, clasificaciones o salidas.

Proceso de reparto:

1. El repartidor inicia sesión en la app móvil.
2. Consulta su ruta del día o la ruta en curso.
3. Inicia la ruta cuando sale a reparto.
4. El sistema activa el seguimiento GPS y comienza a reportar ubicación.
5. En cada parada, el repartidor selecciona la entrega y confirma su resultado.
6. Al finalizar la jornada, el repartidor cierra la ruta con observaciones.

2.3 IMPLEMENTACIÓN

2.3.1 Implementación del frontend con Angular

Las tres aplicaciones frontend se han desarrollado con Angular 21 utilizando componentes standalone y una organización basada en páginas, servicios, guards e interceptores. Esta elección encaja bien con la arquitectura general del proyecto porque evita dependencias innecesarias, reduce el peso del arranque y facilita que cada cliente evolucione de forma aislada.

Cada una de las aplicaciones sigue una misma idea estructural, aunque aplicada a su dominio:

- `app.routes.ts` define la navegación y las pantallas principales.
- `app.config.ts` centraliza proveedores globales, como cliente HTTP, interceptores y configuraciones compartidas.
- Las páginas implementan la lógica de presentación y coordinan formularios, estados visuales y acciones de usuario.
- Los servicios encapsulan el contrato HTTP con el backend y evitan que los componentes conozcan detalles de rutas o payloads.
- Los guards protegen el acceso a rutas privadas y ayudan a restringir el uso por sesión y por rol.
- Los interceptores añaden el token JWT y unifican ciertos comportamientos de comunicación con API.

Una decisión importante del proyecto ha sido no introducir una capa de estado global pesada como NgRx. En su lugar, se ha optado por una combinación de estado local de componente, servicios especializados y señales reactivas en los puntos donde la interfaz necesita reflejar cambios inmediatos. Para el alcance del TFG, esta decisión resulta razonable: mantiene la solución comprensible, evita sobre ingeniería y ofrece suficiente capacidad para modelar formularios multietapa, estados de carga, errores y actualizaciones en tiempo real.

En el caso de la aplicación de clientes, se ha trabajado con servicios específicos para autenticación, envíos, tarifas, pagos, oficinas, perfil y tracking. Esto facilita que cada página consuma exactamente la funcionalidad necesaria y evita duplicar lógica de acceso a API. La experiencia resultante está muy orientada a tarea: contratar un envío, consultar su evolución y mantener la información personal del cliente.

La pantalla de tracking incorpora una combinación de carga inicial por HTTP y suscripción posterior por SignalR. Este diseño evita depender por completo de los eventos en tiempo real y garantiza que el usuario vea de inmediato la información disponible, incluso aunque la conexión WebSocket tarde unos instantes en establecerse. En la intranet y en la app de reparto se repite la misma filosofía: la información crítica se puede obtener por consulta directa, pero se complementa con eventos en vivo cuando estos aportan valor operativo.

La intranet y la app de reparto también siguen un patrón de servicios, aunque adaptado a sus necesidades. En la intranet se concentra la lógica de consultas operativas, dashboards, asignaciones, seguimiento interno, escaneo y notificaciones. En la app de reparto, la lógica gira alrededor de la ruta, las entregas, la geolocalización y la tolerancia a conectividad irregular. Esta consistencia estructural entre frontends facilita el mantenimiento y refuerza la idea de ecosistema de aplicaciones, no de una única interfaz con permisos mezclados.

2.3.2 Aplicación de clientes: funcionalidades implementadas

Registro, login y recuperación de contraseña

El cliente puede registrarse con email, contraseña y nombre completo. Una vez autenticado, el token JWT se almacena localmente y se utiliza para consumir las rutas privadas. El servicio de autenticación del frontend comprueba el rol del usuario y limita el acceso al flujo de cliente, evitando que perfiles internos utilicen por error esta interfaz. El sistema también soporta solicitud de reseteo de contraseña y establecimiento de una nueva contraseña a través de un token enviado por correo, lo que añade una capa de madurez funcional poco habitual en prototipos académicos básicos.

Contratación del envío mediante asistente por pasos

La página de contratación del envío es uno de los elementos más trabajados del proyecto. No se trata de un formulario lineal simple, sino de un asistente por pasos que organiza la información para reducir errores y guiar al usuario durante una operación relativamente compleja.

El flujo se divide en tres fases principales: datos del remitente y oficina de admisión, datos del destinatario y modalidad de entrega, y configuración del paquete con servicio y pago. Esta separación permite validar progresivamente la información y mostrar advertencias en el momento oportuno. La versión actual introduce una regla operativa importante: los envíos contratados online no incluyen recogida a domicilio en origen. El remitente debe entregar el paquete en una oficina postal, mientras que el destinatario puede recibirlo en domicilio o recogerlo en oficina.

Entre las reglas implementadas destacan las siguientes:

- Validación de campos de contacto obligatorios para remitente y destinatario.
- Selección y validación de oficina de origen mediante búsqueda por ciudad o código postal contra el backend.
- Selección de entrega del destinatario en domicilio o en oficina, con su correspondiente validación de dirección o punto de recogida.
- Integración con la agenda de direcciones favoritas para autocompletar datos personales y de entrega.
- Obligación de DNI en operaciones que implican Canarias, reflejando una casuística territorial real.
- Control de peso máximo de 30 kg y verificación de dimensiones compatibles con la operativa.
- Advertencia de recargo cuando la suma de dimensiones supera 210 cm o aparece exceso dimensional relevante.
- Confirmación explícita del precio final calculado por servidor antes de redirigir a Stripe, evitando discrepancias entre lo mostrado y lo realmente cobrado.

El valor de esta implementación no reside solo en la interfaz, sino en la coherencia entre lo que se valida en frontend y lo que finalmente consume backend. El formulario construye un payload estructurado que recoge oficina de origen, modalidad de entrega, datos personales, medidas y servicio, y lo envía al flujo de pago sin introducir reglas comerciales alternativas. De este modo, la experiencia de contratación mantiene consistencia con el motor real de negocio.

Calculadora de tarifas

La calculadora consume directamente el backend y evita realizar estimaciones locales independientes. Esto es importante porque garantiza que la lógica comercial sea única y que el precio mostrado al usuario coincida con el precio que se utilizara en la creación real del envío. El frontend actúa aquí como presentador de resultados, no como segunda fuente de verdad.

El motor de tarifas tiene en cuenta:

- Peso real del paquete.
- Peso volumétrico, calculado a partir de largo, ancho y alto.
- Peso facturable, que es el máximo entre peso real y volumétrico.
- Zona del envío según códigos postales: local, península, Baleares, Ceuta/Melilla o Canarias.
- Tipo de tarifa: estándar o premium.
- Recargo por exceso dimensional cuando la suma de dimensiones supera 210 cm o el lado mayor excede 170 cm.
- IVA del 21 por ciento.

Además de devolver importe, el cálculo también informa de tiempo estimado y condiciones del servicio, lo que ayuda al usuario a comparar opciones antes de pagar. Esto convierte la tarificación en uno de los componentes más realistas del sistema, ya que va más allá de una simple tabla estática y reproduce criterios habituales del sector logístico.

Creación del envío y pasarela de pago

El flujo real implementado no crea el envío como pagado desde el principio. En lugar de eso, al confirmar el formulario se genera en backend un envío en estado `PendientePago`, junto con una sesión de Stripe Checkout. El usuario ve antes de salir de la aplicación un resumen con el importe exacto calculado por

servidor y, solo entonces, es redirigido a la pasarela externa. Una vez completado el pago, el sistema verifica la sesión, marca el envío como pagado, genera documentación y dispara la admisión logística.

La app de clientes contempla además el retorno desde Stripe hacia pantallas de pago exitoso o cancelado. En el escenario de éxito, el frontend consulta al backend para consolidar el resultado real de la sesión. En el escenario de cancelación, el envío no se pierde: permanece como PendientePago y puede reintentarse sin rehacer todo el formulario. Este diseño es importante porque separa con claridad la intención de compra de la confirmación efectiva del pago y evita perder la operación si el usuario abandona temporalmente la pasarela o interrumpe el proceso.

Perfil y agenda de direcciones

El cliente dispone de un panel donde puede:

- Consultar y editar sus datos de perfil ampliado.
- Actualizar los datos de cuenta del sistema de autenticación.
- Cambiar la contraseña desde una vista controlada.
- Crear, editar y eliminar direcciones favoritas.
- Establecer una direccion predeterminada.
- Consultar el listado de envíos asociados a su cuenta.
- Descargar etiquetas y facturas en PDF.

La implementación de este panel es relevante porque unifica varias responsabilidades que, en muchas aplicaciones, aparecen dispersas. Aquí el usuario no solo contrata, sino que mantiene su identidad operativa dentro del sistema, reutiliza datos en futuras contrataciones y accede a la documentación de los envíos ya procesados. Esta parte del proyecto aporta mucha solidez al módulo de cliente, ya que convierte la plataforma en algo más que un simple formulario de contratación puntual.

Tracking público en tiempo real

El tracking publico combina la consulta de trazabilidad existente con la recepción de eventos en vivo. El cliente visualiza una barra de progreso con estados como recogido, oficina de origen, clasificación, transito, CTA destino, oficina destino, reparto y entregado. Además, cuando se producen cambios, estos pueden llegar por SignalR con información de estado, entrega, incidencia o ubicación.

Desde el punto de vista de implementación, esta pantalla no se limita a pintar un texto de estado. Incluye una traducción de la realidad interna a una narrativa comprensible para el cliente, gestiona la conexión y reconexión al hub, controla suscripciones por número de seguimiento y actualiza visualmente el progreso del envío. Esta funcionalidad es especialmente valiosa porque aporta una sensación de sistema vivo y elimina la necesidad de recargar continuamente la página para ver novedades.

2.3.3 Microservicio de autenticación

El microservicio Auth se ha implementado con ASP.NET Core Identity, utilizando un modelo de usuario propio que extiende IdentityUser con nombre completo, código de empleado, fecha de registro y rol. Sobre esta base se construyen los procesos de login, registro, consulta del usuario autenticado, actualización del perfil de cuenta, cambio de contraseña, refresh token y reseteo de contraseña por correo.

Desde el punto de vista de arranque, el servicio configura conexión a PostgreSQL, autenticación JWT, Identity y políticas de autorización, y además aplica migraciones y prepara datos base necesarios para que el sistema pueda arrancar de forma consistente. Esto es importante porque evita depender de configuraciones manuales posteriores al despliegue.

En la capa de API, el AuthController concentra endpoints de alto valor funcional. No solo permite autenticar y registrar usuarios, sino también recuperar el contexto del usuario autenticado, modificar información de cuenta, cambiar la contraseña, emitir nuevos tokens mediante refresh y lanzar el flujo de recuperación de acceso. Esta amplitud funcional convierte a Auth en un servicio transversal real y no en un simple endpoint de login.

Desde el punto de vista técnico, la autenticación utiliza JWT con validación de firma, emisor, audiencia y expiración sin margen adicional de cinco minutos, lo cual endurece el comportamiento de la sesión.

Asimismo, el sistema soporta refresh tokens reales con rotación, expiración y revocación, lo que mejora la experiencia de usuario y acerca la aplicación a una implementación profesional.

El módulo Auth es compartido por todas las aplicaciones del sistema, pero cada frontend aplica restricciones propias sobre el rol del usuario que puede entrar. Por ejemplo, la app de clientes limita el acceso a usuarios con rol Cliente, mientras que la intranet y la app de reparto esperan perfiles internos. De este modo, la identidad es común, pero el contexto de uso no lo es.

2.3.4 Microservicio ciudadano

El módulo Ciudadano es el núcleo del dominio funcional visible para el cliente. En él se concentra la gestión de envíos, el cálculo de tarifas, los pagos, la generación de documentos, el perfil del cliente, el directorio de oficinas y el tracking. Es, por tanto, el servicio con mayor densidad de reglas de negocio orientadas a contratación y seguimiento.

Su implementación presenta varios puntos de interés.

A) Gestión de envíos

El controlador de envíos expone endpoints para cotizar, crear envíos, consultar tracking, recuperar envíos del usuario y obtener documentos asociados. Cuando se crea un envío, se generan dos identificadores: un numero de seguimiento público y un numero de expedición interno. También se fija un estado público y un estado interno inicial, lo que permite mantener dos niveles de visibilidad diferentes sobre el mismo objeto de negocio.

El mismo controlador incluye endpoints internos para que otros servicios publiquen cambios de ubicación o eventos de entrega. Este detalle es importante porque sitúa a Ciudadano como dueño del tracking visible para el cliente, incluso cuando la información original procede de Reparto. En otras palabras, Reparto informa de la operativa, pero Ciudadano decide como esa operativa se persiste y se expone públicamente.

B) Oficinas postales

La gestión de oficinas ha dejado de apoyarse exclusivamente en un JSON estático consumido por el frontend. En la versión actual, Ciudadano expone endpoints de listado y búsqueda que devuelven oficinas persistidas y preparadas para ser consumidas por la web pública. El cliente carga ese catalogo desde backend, lo transforma a su modelo de interfaz y mantiene una cache en memoria para agilizar sugerencias y autocompletado. Paralelamente, la intranet mantiene el maestro operativo de oficinas y permite administrarlo desde paneles de backoffice.

El resultado práctico es un modelo híbrido: se siguen utilizando fuentes estructuradas y seeders para inicializar o resolver información geográfica, pero la experiencia de usuario y la operativa interna ya se apoyan en APIs y entidades persistidas, no en un archivo local estático inmutable.

C) Motor de tarifas

El cálculo de precios se concentra en un servicio dedicado de tarifas. Su responsabilidad va más allá de devolver un importe: resuelve la zona del envío, calcula el peso volumétrico, determina el peso facturable, aplica recargos dimensionales, diferencia entre servicio estándar y premium, añade IVA y devuelve una estimación temporal del servicio. Esta centralización evita incoherencias entre pantallas y asegura que cualquier proceso que necesite precio utilice la misma lógica comercial.

D) Pagos y documentos

La integración con Stripe se articula en un flujo completo:

1. El usuario autenticado solicita crear una sesión de pago.
2. El backend crea un envío en estado pendiente de pago.
3. Se genera una sesión de Stripe Checkout con URL de éxito y cancelación.
4. Tras el pago, el sistema verifica la sesión, por retorno del cliente o por webhook.
5. Si el pago es correcto, actualiza el envío, genera la etiqueta y la factura, y envía un correo de confirmación.

Este proceso representa muy bien una integración empresarial real. No se trata solo de "cobrar", sino de coordinar un conjunto de acciones dependientes del resultado del pago. El PagosController soporta creación de sesión, verificación posterior y reintento de pagos pendientes, mientras que el StripeService encapsula la comunicación con la API externa. Cuando el pago se consolida, el sistema ejecuta un procesamiento adicional que genera documentos PDF, actualiza estados, registra fechas y notifica a logística que el paquete debe ser admitido. A partir de ese punto, la última milla no se autoasigna desde Ciudadano: el envío entra en el circuito logístico y solo pasa a reparto cuando la clasificación interna lo libera como DisponibleParaReparto.

Además, el sistema incorpora un servicio en segundo plano que revisa periódicamente pagos pendientes para detectar confirmaciones que no hayan quedado correctamente reflejadas solo con el retorno del navegador. Esta capa aporta robustez y reduce el riesgo de inconsistencias en escenarios reales de navegación o red.

E) Tracking y SignalR

El módulo ciudadano incorpora un TrackingHub público al que los clientes se suscriben por número de seguimiento. Los eventos principales emitidos son:

- EstadoActualizado.
- UbicacionActualizada.
- EntregaCompletada.
- IncidenciaDetectada.

La aplicación cliente escucha estos eventos para refrescar visualmente la trazabilidad. El hub organiza suscripciones por grupos del tipo tracking-{numero}, lo que permite aislar los eventos de cada envío. Este mecanismo sirve de puente entre la operativa interna y la experiencia pública del usuario, y se complementa con un servicio de notificación que abstrae la emisión de eventos para no acoplarla a un único controlador.

F) Sincronización con reparto

Ciudadano expone endpoints internos para que el módulo de reparto publique ubicaciones o notifique eventos de entrega. Cuando reparto comunica que un envío ha sido entregado, rechazado, intentado o devuelto, el módulo ciudadano transforma ese evento en un estado interno y un estado público coherentes, actualiza la base de datos y emite las notificaciones SignalR correspondientes.

Este punto es de gran relevancia porque demuestra una integración real entre dominios. La aplicación no se limita a tener una pantalla de tracking decorativa, sino que enlaza el resultado operativo del repartidor con la información que ve el cliente. De hecho, una de las claves del diseño es que el estado público nunca se actualiza de forma arbitraria desde el frontend: siempre procede de procesos reales de negocio consolidados en backend.

2.3.5 Microservicio de intranet y logística

El módulo intranet modela la red logística interna. Sus procesos principales son la admisión, la resolución de CTAs, la creación de movimientos, la asignación de tareas, la gestión de incidencias, el soporte al escaneo operativo y el backoffice administrativo. Desde el punto de vista arquitectónico, este servicio es el puente entre la contratación del envío y la realidad física de la red logística.

A) CTAs y red logística

Los CTAs se representan como nodos principales de la red. Cada uno tiene código, nombre, área zonal, provincia, ciudad, dirección, código postal y capacidades como nodo aéreo o marítimo. La versión actual no se limita a consultar estos nodos: permite administrarlos desde la intranet, incluyendo creación, edición, activación y desactivación. Además, existen dashboards por CTA y una vista global administrativa con métricas agregadas de tareas, incidencias, movimientos y operarios.

La resolución del CTA de destino se apoya en el código postal. Cuando se admite un paquete, el sistema determina a que centro debe dirigirse y, si el origen y el destino corresponden a CTAs distintos, crea

automáticamente un movimiento troncal. En otras palabras, la admisión no se limita a registrar que el paquete existe: decide en que punto de la red debe entrar y como debe comenzar a desplazarse.

B) Admisión de paquetes y alta presencial

La admisión es uno de los procesos mejor definidos del proyecto. Su flujo base es el siguiente:

1. Se recibe una solicitud con los datos del paquete y del destino.
2. Se resuelve el CTA de destino según el código postal.
3. Si el paquete debe viajar a otro nodo, se calcula el tipo de transporte más adecuado y se crea un movimiento troncal.
4. Se registran eventos de historial y se notifica en tiempo real a los actores operativos.
5. El paquete entra en el circuito de tareas y movimientos internos hasta que la clasificación lo libere para reparto o para entrega en oficina.

La diferencia importante respecto a versiones anteriores es que la admisión ya no autoasigna directamente una ruta de reparto. El `AdmisionService` registra y orquesta la entrada del paquete en la red, pero la última milla se materializa más tarde, cuando un escaneo logístico marca el envío como `DisponibleParaReparto`. Solo en ese momento se informa al dominio de reparto y el `JefeReparto` decide a que ruta incorporarlo.

El mismo servicio soporta también el alta presencial en oficina. Esta pantalla permite que un `OperarioOficina` cree un envío cuando el cliente lleva físicamente el paquete a ventanilla, elija el método de cobro, busque una oficina de destino si la entrega es en oficina y obtenga como resultado número de seguimiento, número de expedición, coste calculado y CTA destino. Con ello, la intranet no solo consume envíos pagados desde la web publica, sino que también genera operativa directamente desde el punto físico.

C) Asignaciones, tareas y cadena de escaneo

Las asignaciones representan tareas atómicas sobre expediciones concretas dentro de un CTA o una oficina. Cada tarea tiene un tipo, un estado, un responsable, un creador, una fecha de asignación y, en su caso, fechas de inicio y finalización. Con esta estructura, la intranet no solo conoce que un paquete existe, sino también que trabajo físico debe realizarse sobre él y quien debe hacerlo.

La aplicación interna permite crear asignaciones, filtrarlas por estado y ejecutar transiciones de inicio, finalización, cancelación y reasignación. En la versión actual, `Admin` y `Supervisor` pueden intervenir administrativamente sobre la cola, mientras que `OperarioCTA` y `OperarioOficina` trabajan sobre sus pendientes y en progreso. Además, el sistema conserva completadas recientes y permite limpiar visualmente la vista sin borrar la evidencia del backend.

Una mejora especialmente relevante es el encadenamiento por escaneo. Cada `Handler` del `ScanProcessorService` no solo valida el código, sino que cierra la tarea actual y crea la siguiente cuando procede. Esto permite articular un recorrido operativo coherente, por ejemplo: recepción en oficina, salida hacia CTA, recepción en CTA, clasificación, despacho troncal, recepción troncal y liberación para reparto. El estado `DisponibleParaReparto` actúa como frontera entre logística interna y última milla.

D) Incidencias, movimientos y seguimiento interno

El módulo intranet gestiona tanto incidencias como movimientos troncales. Las incidencias pueden filtrarse por estado, CTA y tipo, y entre los tipos implementados aparece `PaqueteFueraDeTareas`, que resuelve una casuística muy real: encontrar un paquete que no encaja con la cola del operario que lo está manipulando. Los movimientos globales permiten visualizar expediciones entre CTAs, su transporte, estado y urgencia, lo que aporta una capa de supervisión transversal sobre la red logística.

Además, la pantalla de seguimiento interno permite buscar expediciones por número de seguimiento o por número de expedición y consultar el historial detallado. Este punto es crucial porque demuestra que el sistema distingue claramente entre visibilidad para cliente y control de la operativa interna.

E) Notificaciones internas y backoffice administrativo

El módulo intranet incorpora `SignalR` para notificaciones en tiempo real dirigidas a CTAs y perfiles concretos. De esta manera, eventos como la recepción de un paquete, la asignación de una tarea, la llegada de un movimiento o la disponibilidad de un paquete para reparto pueden llegar directamente a los usuarios que

deben actuar sobre ellos. Esta estrategia reduce la dependencia de refrescos manuales y refuerza la sensación de sistema operativo vivo.

Sobre esa base operativa se ha construido un backoffice administrativo mucho más amplio que en una primera interacción del proyecto. El rol Admin dispone de módulos para gestionar usuarios internos, clientes, envíos, repartidores, oficinas, CTAs, tarifas, incidencias globales, movimientos globales y broadcast de notificaciones. Esto convierte la intranet en una plataforma de explotación real del sistema y no solo en una interfaz de escaneo.

F) integración con reparto

La integración con reparto ya no se entiende como una autoasignación inmediata desde la admisión. En la versión actual, el evento técnico decisivo es el escaneo DisponibleParaReparto. Ese Handler registra el evento de historial, emite notificaciones operativas y añade el paquete a la bandeja persistente del microservicio de reparto. A partir de ahí, el JefeReparto decide si crea una ruta nueva o si incorpora el paquete a una ruta ya planificada. Este desacoplamiento refleja mejor la realidad logística: clasificación y última milla pertenecen a dominios distintos y no siempre deben resolverse en el mismo instante.

2.3.6 Microservicio de reparto

El módulo de reparto se encarga de la última milla, es decir, del tramo final del envío entre la oficina de destino y el destinatario. Incluye el modelo de repartidor, la definición de rutas diarias, las entregas asociadas a cada ruta, la ubicación GPS y, en la versión actual, también las superficies de gestión del JefeReparto. Su importancia dentro del proyecto es alta porque conecta el mundo de la planificación interna con la ejecución en movilidad.

A) Repartidores y perfiles operativos

Cada repartidor se asocia a un usuario del sistema y a una oficina de referencia. Esto permite enlazar sus credenciales de acceso con su información operativa y con el contexto físico desde el que trabaja. El módulo soporta dos perfiles: Repartidor, orientado a ejecución de ruta, y JefeReparto, orientado a coordinación y supervisión. Ambos se apoyan en la misma base de dominio, pero no consumen los mismos endpoints ni la misma experiencia de usuario.

El microservicio expone endpoints para recuperar el perfil del usuario autenticado, listar repartidores, editar sus datos operativos, activarlos o desactivarlos y consultar sus rutas del día. Esto permite que la gestión del equipo de última milla ya no dependa solo del backoffice general de la intranet, sino también de herramientas propias del área de reparto.

B) Rutas de reparto y entregas

Las rutas se identifican con códigos del tipo REP-YYYYMMDD-XXX, se asignan a una fecha y a un repartidor concreto, y agrupan varias entregas. Una ruta puede estar planificada, en curso o completada, y conserva información como hora de salida, hora de regreso y observaciones generales.

La propia ruta actúa como unidad de trabajo diaria, permitiendo resumir progreso, volumen de entregas y resultados de la jornada. Desde el backend, el servicio de reparto soporta carga de la ruta activa, inicio de jornada, finalización y consulta de entregas asociadas, de forma que el frontend móvil puede reconstruir el contexto completo de la operativa del día.

Cada entrega modela una parada real de la ruta y contiene información detallada sobre dirección, código postal, ciudad, destinatario, teléfono, intento, orden en la ruta, estado, fecha, receptor, coordenadas, firma y foto. Esta riqueza de datos hace que el sistema sea apto tanto para el seguimiento operativo como para la aportación de evidencias.

C) Ubicación, GPS y trazabilidad de última milla

El módulo de reparto no se limita a gestionar estados. También expone un endpoint para registrar ubicación, mantiene histórico operativo de posiciones y publica vistas de ubicaciones activas para supervisión en tiempo real. Gracias a esto, la posición del repartidor puede aprovecharse en dos

direcciones: como herramienta de control para el JefeReparto y como fuente de eventos para enriquecer el tracking ciudadano.

Esta relación entre servicios es especialmente relevante porque demuestra que Reparto no es un módulo aislado de movilidad, sino una fuente activa de información para el resto del sistema. El backend de reparto consolida los datos de la ejecución de ruta y los transforma en eventos consumibles por el dominio ciudadano.

D) Bandeja del JefeReparto y asignación manual a rutas

Una de las evoluciones más importantes del proyecto es la introducción de una bandeja persistente de paquetes pendientes de reparto. Cuando Intranet marca un envío como DisponibleParaReparto, el paquete se registra en PaquetePendienteReparto y queda visible para el JefeReparto. Desde esa bandeja puede:

- Seleccionar uno o varios paquetes listos para salir a calle.
- Crear una ruta nueva para un repartidor concreto.
- Añadir paquetes a una ruta ya planificada.
- Reasignar entregas entre rutas del mismo día cuando cambian las necesidades operativas.

Este modelo sustituye la idea de autoasignación directa desde admisión por una planificación manual asistida, mucho más realista en un contexto operativo donde la disponibilidad de personal, la carga diaria y la urgencia influyen en cada decisión.

E) Gestión del equipo de reparto

El módulo incluye también endpoints y vistas para listar repartidores, editar su teléfono o vehículo, desactivarlos cuando no deben operar y reactivarlos después. En el caso del JefeReparto, el backend filtra automáticamente por su propia oficina, de manera que solo gestiona el equipo sobre el que realmente tiene responsabilidad operativa.

Esto convierte a Reparto en un dominio con autonomía real: no solo ejecuta entregas, sino que permite organizar personas, rutas y paradas desde su propio contexto de negocio.

F) Sincronización con Ciudadano

La confirmación de entrega no se reduce a cambiar un estado booleano. El servicio admite distintos resultados operativos, observaciones y pruebas asociadas, y a continuación informa a Ciudadano para consolidar el tracking público. De esta manera, la app del cliente refleja lo que ha ocurrido realmente en calle y no una simulación independiente. Esta sincronización entre dominios es una de las mejores muestras de madurez arquitectónica del sistema.

2.3.7 Aplicación de reparto: operativa móvil, jefe de reparto y soporte offline

La aplicación de repartidores es probablemente una de las piezas más avanzadas del proyecto desde el punto de vista técnico, porque combina operativa de negocio con problemas propios de movilidad.

A) Repartidor: ruta activa y operativa de calle

Para el perfil Repartidor, la aplicación gira alrededor de la ruta activa del día. La pantalla principal muestra el código de ruta, estado, progreso, entregas completadas, pendientes y fallidas, junto con la siguiente parada recomendada. Desde ella se pueden iniciar y finalizar rutas, seleccionar entregas concretas, abrir navegación externa y registrar el resultado de cada parada.

Este enfoque convierte la app en una herramienta de ejecución directa: no es un visor pasivo, sino el puesto de trabajo móvil del repartidor.

B) Seguimiento GPS

Cuando una ruta pasa a estado En Curso, la app activa el seguimiento GPS. Para ello combina varias estrategias:

- watchPosition para recibir actualizaciones continuas del navegador.
- Envío inicial al empezar la ruta.
- Heartbeat periódico para mantener actividad, aunque no haya cambios frecuentes.

- Ajuste del comportamiento cuando la app entra en segundo plano.
- Reintentos progresivos si fallan las comunicaciones.

La implementación incorpora además control del ciclo de vida de la página, reactivación del seguimiento al volver a primer plano y gestión de temporizadores auxiliares. Esta lógica demuestra que no se ha tratado la geolocalización como una simple llamada puntual, sino como un proceso continuo con consideraciones de rendimiento, batería y conectividad.

C) Cola offline

La app mantiene una cola en localStorage para dos tipos de eventos: confirmaciones de entrega y ubicaciones pendientes. Si el dispositivo pierde conexión, los datos no se descartan. En su lugar, quedan encolados y se reintentan cuando vuelve la conectividad. Esta decisión es clave en un contexto de reparto, donde el uso en movilidad hace que la cobertura no siempre sea estable.

La existencia de este servicio offline convierte a la app en una herramienta utilizable en condiciones reales de campo. Incluso aunque no exista sincronización completa bidireccional ni una base local avanzada, la solución logra preservar las acciones más críticas para que la jornada no quede bloqueada por una pérdida temporal de red.

D) Mapa, navegación y confirmación de entrega

La vista de ruta integra Leaflet y OpenStreetMap para representar:

- La posición actual del repartidor.
- El historial reciente de la traza GPS.
- Los puntos de entrega con coordenadas disponibles.
- La entrega seleccionada o la siguiente parada.

Además, se ofrecen enlaces rápidos a Google Maps y Waze para facilitar la navegación externa. Aunque esto no equivale a un navegador embebido completo, proporciona una solución útil y realista dentro del alcance del proyecto. La decisión de apoyarse en Leaflet y OpenStreetMap permite disponer de cartografía funcional sin depender de servicios comerciales cerrados para la visualización principal.

E) Confirmación de entrega con evidencias

La app permite confirmar distintas situaciones: entrega correcta, entrega en punto alternativo, ausente, dirección incorrecta, rechazo o devolución a oficina. Junto a ese estado, el repartidor puede registrar el nombre del receptor, su DNI, observaciones, firma digital, foto y coordenadas. Todo ello convierte la confirmación de entrega en un registro mucho más rico que un simple cambio de estado.

La aplicación incluye además una pantalla de escaneo operativo, capaz de consultar expediciones y sugerir la siguiente acción interna. Esto conecta la actividad del repartidor con la lógica de estados del sistema y evita tratar la app móvil como una simple lista de paradas.

F) Jefe de reparto: bandeja, rutas y supervisión

El perfil JefeReparto dispone de una experiencia propia dentro de la misma aplicación. Sus herramientas principales son:

- Dashboard con métricas del día, como rutas en curso, entregas pendientes, entregas fallidas y repartidores activos.
- Bandeja de paquetes disponibles para reparto, desde la que puede crear rutas o añadir paquetes a rutas planificadas.
- Gestión de rutas del día, con visibilidad sobre reparto asignado y progreso.
- Reasignación de paradas entre rutas.
- Mapa en tiempo real con ubicaciones activas del equipo.
- Gestión de sus repartidores, con capacidad de editar datos operativos y activar o desactivar perfiles.

Esta parte de la aplicación tiene mucho valor porque desplaza la coordinación de la última milla al mismo ecosistema en el que luego se ejecuta. No hace falta abrir una cuarta aplicación ni volver a la intranet para resolver decisiones diarias de reparto.

2.3.8 API Gateway y control de acceso

El API Gateway centraliza el acceso desde los distintos frontends y sirve como punto único de entrada a la capa de microservicios. Entre sus funciones destacan:

- Aplicar CORS de forma centralizada.
- Integrar autenticación JWT.
- Determinar que rutas son públicas y cuales protegidas.
- Orquestrar el enrutamiento hacia el backend correspondiente.

Una decisión destacable es la definición explícita de rutas públicas para login, registro, refresh, recuperación de contraseña, consulta de tracking, consulta de tarifas, webhook de Stripe y búsqueda de oficinas. El resto de las rutas queda protegido por defecto. De este modo, el gateway refleja claramente la frontera entre lo que debe estar abierto al exterior y lo que requiere contexto autenticado.

Esta pieza tiene un valor especial dentro de la arquitectura porque evita que cada microservicio tenga que replicar exactamente la misma lógica de entrada externa. Aunque los servicios mantienen su propia seguridad interna, el gateway concentra la política de exposición pública y simplifica a los frontends el consumo de APIs. También facilita una futura evolución de la plataforma, por ejemplo, si se desea introducir limitación de peticiones, trazas centralizadas o políticas más avanzadas de observabilidad y seguridad en el punto de entrada.

2.4 IMPLANTACIÓN, DESPLIEGUE E INSTALACIÓN

2.4.1 Entorno de desarrollo local

Para desarrollo local, Nexopostal se apoya en Docker Compose. Esto permite que cualquier entorno con Docker pueda levantar toda la plataforma sin necesidad de instalar manualmente cada dependencia por separado. La decisión es especialmente útil en un proyecto con tres frontends, varios backends y varias bases de datos, donde la puesta en marcha manual sería mucho más propensa a errores.

La configuración local incluye:

- Proxy Nginx sin SSL.
- API Gateway.
- Tres aplicaciones Angular.
- Cuatro microservicios principales.
- Cuatro bases de datos PostgreSQL.

En la práctica, docker-compose.yml ofrece un entorno completo de extremo a extremo para desarrollo local. La web de clientes, la intranet y la app de reparto se sirven a través del proxy local, mientras que gateway, microservicios y bases de datos conviven en una misma red Docker. Esto permite probar flujos completos como registro, contratación, pago, admisión, clasificación, liberación a reparto, tracking y entrega sin necesidad de cambiar manualmente URLs ni levantar componentes uno a uno.

Conviene señalar además que el flujo local actual usa directamente docker-compose.yml como compose por defecto mediante `docker compose up -d --build`. La variante `docker-compose.local.yml` ya no forma parte del circuito operativo vigente, lo que simplifica la puesta en marcha y evita duplicidades de configuración.

El hecho de levantar la plataforma completa desde un único archivo simplifica la puesta en marcha y reduce el coste de incorporación de nuevos desarrolladores o de nuevas pruebas en otras máquinas. Además, mejora la paridad con producción, ya que la topología general de servicios se mantiene.

2.4.2 Configuración en contenedores

Cada frontend dispone de su propio Dockerfile, al igual que cada microservicio .NET. Esta decisión permite construir imágenes independientes y desplegarlas de forma desacoplada. No todos los componentes tienen el mismo ciclo de cambio ni las mismas necesidades de recursos, por lo que empaquetarlos por separado resulta coherente con la arquitectura general.

Las variables de entorno se inyectan mediante archivos .env y configuración jerárquica por servicio, lo que facilita adaptar el comportamiento entre desarrollo y producción sin modificar el código fuente. Gracias a ello, la misma solución puede conectarse a bases de datos distintas, cambiar secretos JWT, activar configuraciones de Stripe o variar URLs internas sin recompilar toda la plataforma.

El uso de contenedores aporta varias ventajas:

- Reproducibilidad del entorno.
- separación clara entre servicios.
- fácil integración con pipelines CI/CD.
- Mayor control sobre dependencias y puertos.
- Posibilidad de actualizar o reiniciar módulos concretos sin afectar a toda la plataforma.

2.4.3 Despliegue en producción

En producción, el sistema se despliega en un VPS, utilizando imágenes publicadas en GHCR. El despliegue contempla Nginx con SSL, sin exposición directa de las bases de datos, y con todos los servicios conectados a una red interna. Esta separación entre compilación y despliegue es importante: el servidor no necesita construir el proyecto, sino solo descargar imágenes ya validadas y ejecutar la nueva versión mediante un flujo de pull y up -d --no-build sobre docker-compose.production.yml.

La publicación se realiza bajo varios dominios y subdominios, con una separación clara entre clientes, intranet y reparto. Los certificados del proxy deben provisionarse previamente en el servidor para que la entrada HTTPS funcione correctamente, aunque el pipeline contempla la generación de un certificado temporal autofirmado como salvaguarda inicial. Esto refleja una necesidad operativa real de cualquier despliegue web serio.

Este modelo es adecuado para un proyecto TFG porque combina realismo técnico con una infraestructura asumible. No requiere una plataforma cloud compleja, pero al mismo tiempo obliga a trabajar con dominios, certificados, registros, variables seguras y coordinación de servicios.

2.4.4 integración y despliegue continuo

El proyecto incluye un workflow de GitHub Actions que automatiza buena parte del ciclo de entrega. Cuando se produce un push sobre ramas relevantes, el pipeline:

1. Compila los microservicios .NET.
2. Compila las aplicaciones Angular.
3. Construye las imágenes Docker de todos los servicios.
4. Ejecuta los test de la aplicación, y si falla alguno para el despliegue.
5. Publica dichas imágenes en GitHub Container Registry.
6. En rama master, conecta con el VPS, publica el .env, sincroniza la configuración necesaria y ejecuta el despliegue por pull y compose up -d --no-build.

El valor de este pipeline no es solo técnico, sino metodológico. El proyecto no termina cuando el código compila en local, sino cuando existe un proceso repetible para construir, publicar y desplegar una versión. Además, el uso de GHCR reduce el tiempo de despliegue en el VPS, ya que este solo necesita descargar imágenes y relanzar los contenedores necesarios.

Esta automatización tiene un valor muy importante dentro del TFG porque demuestra una visión completa del ciclo de vida del software. No se entrega solo código fuente, sino una solución preparada para integración continua y despliegue automatizado.

2.4.5 Configuración de red y proxy

Nginx actúa como reverse proxy, resolviendo que dominio o subdominio debe servir cada aplicación. Esto permite exponer, por ejemplo, una URL para clientes, otra para intranet y otra para driver. A nivel conceptual, esta separación mejora la organización del sistema y se alinea con la segmentación funcional ya presente en el resto de la arquitectura.

La configuración del proxy es relevante no solo por el enrutado HTTP tradicional, sino también por el soporte a conexiones WebSocket, necesarias para los hubs de SignalR. Además, el despliegue incorpora una mitigación frente al cacheo de DNS interno de Docker mediante el uso de un resolver explícito, evitando problemas de cruce entre dominios o backends tras recrear contenedores. Este tipo de ajuste demuestra que la implantación no se ha quedado en un ejemplo teórico, sino que ha tenido en cuenta incidencias reales de operación.

2.5 DOCUMENTACIÓN

La documentación final del proyecto no se limita a esta memoria principal. En la versión actual se articula en tres piezas coordinadas: la memoria del TFG como documento troncal, un manual técnico como anexo para instalación, arquitectura y mantenimiento, y un manual de usuario como anexo para la explotación funcional por perfiles.

2.5.1 Documentación técnica del código

La documentación técnica del proyecto se apoya en varios niveles:

- Un README general del repositorio que resume arquitectura, módulos, roles, patrones de error y formas de despliegue.
- Documentos auxiliares de análisis funcional, flujos logísticos y análisis de roles.
- Comentarios XML y comentarios técnicos en controladores, hubs y servicios del backend.
- Estructura del repositorio organizada por aplicaciones y microservicios.
- Un anexo específico de manual técnico, pensado para instalación, operación y mantenimiento del sistema.

En los backends .NET se ha trabajado con comentarios que describen el objetivo de controladores, endpoints, servicios y hubs, lo cual facilita la comprensión del sistema y permite generar documentación complementaria mediante Swagger u otras herramientas. La memoria, por su parte, sintetiza las decisiones de diseño; el manual técnico desciende al nivel operativo y de soporte.

2.5.2 Documentación de API

Los microservicios principales cuentan con configuración de Open API o Swagger, lo que permite inspeccionar contratos, endpoints y mecanismos de autenticación. Este aspecto es especialmente útil para depuración, pruebas e integración entre equipos. A ello se suma la documentación del gateway y del patrón uniforme de errores, que aporta una capa adicional de claridad sobre cómo se exponen realmente las APIs al exterior.

Los endpoints más relevantes del sistema son:

En Ciudadano:

- Cotización y creación de envíos.
- Consulta de tracking público.
- Consulta de envíos del usuario.
- Descarga de etiqueta y factura.
- Consulta y actualización de perfil.
- Endpoints internos de tracking.

En Auth:

- Login, registro y refresh.
- Consulta del usuario autenticado.
- Recuperación y reseteo de contraseña.

En Intranet:

- admisión de paquetes.
- Asignaciones.
- Movimientos.

- Incidencias.

En Reparto:

- Perfil del repartidor.
- Ruta del día.
- Rutas del día y planificación.
- Entregas.
- Confirmación de entrega.
- Registro de ubicación.
- Bandeja del jefe de reparto.
- Reasignación de entregas y gestión de repartidores.

2.5.3 Documentación de usuario

La propia estructura de las tres aplicaciones funciona como base para un manual de usuario, pero en la versión actual se ha considerado más adecuado separar ese contenido en un anexo específico de manual de usuario. Ese documento organiza la operativa por perfil y por aplicación, de modo que cada actor consulte solo la parte que necesita: cliente, operario de oficina, operario CTA, supervisor, administrador, repartidor o jefe de reparto.

De cara a la entrega final en Word, esta sección y el anexo correspondiente deberían acompañarse con capturas de pantalla debidamente numeradas y referenciadas en el texto. Algunas especialmente recomendables son:

- Pantalla de inicio de clientes.
- Wizard de nuevo envío en sus tres pasos.
- Tracking público con barra de progreso.
- Panel de usuario con agenda de direcciones.
- Dashboard de intranet por rol.
- Gestión de CTA.
- Pantalla de asignaciones y escaneo integrado.
- Alta presencial en oficina.
- Dashboard administrativo.
- Dashboard del jefe de reparto.
- Vista de ruta con mapa y lista de entregas.
- Bandeja de paquetes para reparto.

2.5.4 Organización del repositorio

El repositorio está organizado por áreas funcionales, lo cual facilita localizar el código y comprender la arquitectura.

- clientes-app: frontend de clientes.
- intranet-app: frontend de operativa interna.
- driver-app: frontend de reparto.
- microservicios/Nexopostal/Nexopostal.Auth: autenticación.
- microservicios/Nexopostal/Nexopostal.Ciudadano: dominio ciudadano.
- microservicios/Nexopostal/Nexopostal.Intranet: dominio logístico.
- microservicios/Nexopostal/Nexopostal.Reparto: dominio de reparto.
- microservicios/Nexopostal/Nexopostal.Gateway: gateway.
- nginx: configuraciones del proxy.
- docker-compose*.yaml: escenarios de ejecución y despliegue.

Esta organización refuerza la mantenibilidad del proyecto y hace visible, desde la propia estructura de carpetas, la modularidad buscada durante el desarrollo.

3 RESULTADOS Y DISCUSION

El resultado final del proyecto puede considerarse claramente satisfactorio en relación con los objetivos planteados al inicio. Se ha conseguido construir una plataforma funcional, compuesta por varias aplicaciones y varios servicios, que reproduce de forma realista los procesos esenciales de una empresa de paquetería: contratación del envío, pago, admisión logística, reparto y seguimiento público.

La evaluación de los resultados puede hacerse en varios planos.

En el plano funcional, el sistema cubre un ciclo de negocio completo. El cliente puede registrarse, autenticarse, calcular tarifas, contratar un envío, pagar mediante Stripe, consultar su historial, descargar documentos y seguir la evolución del paquete. Paralelamente, la intranet permite admitir el paquete en la red, resolver CTAs, crear tareas, trabajar con escaneo y gestionar seguimiento interno. Finalmente, el módulo de reparto permite gestionar una bandeja de pendientes, planificar o reutilizar rutas, registrar entregas, emitir ubicaciones y alimentar el tracking visible para el cliente. En el plano arquitectónico, el proyecto demuestra que la separación por dominios no se ha quedado en una decisión superficial. Cada microservicio tiene responsabilidades razonablemente bien delimitadas, cada frontend responde a un perfil de usuario distinto y la integración entre módulos se apoya en contratos claros. La existencia de dos identificadores por envío, la separación entre estado interno y estado público, y la combinación de HTTP con SignalR son ejemplos concretos de decisiones de diseño que aportan coherencia y valor.

En el plano técnico, el trabajo alcanza un nivel superior al de una aplicación académica elemental. No solo se han implementado CRUDs o pantallas simples, sino integraciones con Stripe, emisión de documentos, autenticación JWT con refresh tokens, colas offline en movilidad, notificaciones en tiempo real, despliegue con Docker y Nginx, y automatización CI/CD mediante GitHub Actions. Este conjunto de piezas hace que el proyecto tenga un grado apreciable de realismo.

En el plano operativo, también se han obtenido resultados relevantes. La plataforma puede levantarse en local con todos sus componentes, puede desplegarse en un VPS con imágenes en GHCR y cuenta con una base de monitorización ampliable. Esto demuestra que el trabajo no se ha quedado en el desarrollo aislado del código, sino que ha considerado el ciclo de vida completo de la solución. Uno de los logros más importantes del proyecto es la coherencia entre capas. La interfaz del cliente no está desconectada del backend, el backend ciudadano no está aislado de la intranet, y el reparto no funciona como una demo separada, sino como una pieza que participa activamente en el ciclo completo del envío. Esta continuidad funcional es probablemente el mejor indicador de madurez del sistema.

No obstante, también es importante discutir las limitaciones y puntos de mejora. Aunque la arquitectura es adecuada para el alcance del TFG, la comunicación entre microservicios todavía puede robustecerse. Actualmente se ha implementado seguridad interservicio mediante X-Service Key y sincronización por llamadas HTTP, pero en un escenario de mayor volumen sería recomendable introducir patrones de mensajería más resistentes a fallos transitorios, como outbox/inbox, de duplicación por identificador de evento y trazabilidad distribuida.

Otra limitación está en la parte de reparto. La aplicación móvil web incorpora un seguimiento GPS trabajado y una cola offline útil, pero sigue dependiendo de las capacidades del navegador y del sistema operativo. Esto significa que la experiencia en segundo plano y la estabilidad de la geolocalización no alcanzan todavía el nivel de una aplicación nativa especializada.

También puede mejorarse la parte de navegación de rutas. Actualmente se ofrecen enlaces a aplicaciones externas como Google Maps o Waze, lo cual es práctico y suficiente para el alcance actual, pero no equivale a disponer de un motor embebido con ETA, recalcular o asistente de

conducción integrado.

Desde la perspectiva operativa, existe además una dependencia clara de la infraestructura del servidor en aspectos como el correo SMTP o la provisión de certificados. Esto no invalida el proyecto, pero si recuerda que una solución completa requiere no solo buen código, sino también condiciones de operación adecuadas.

En resumen, el proyecto ofrece resultados claramente positivos. No solo cumple su función principal, sino que además demuestra madurez de diseño, coherencia entre módulos y un nivel técnico superior al de una aplicación académica elemental. Al mismo tiempo, conserva un margen de mejora realista y bien identificado, lo cual refuerza la credibilidad de la memoria

4 TRABAJO FUTURO (OPCIONAL)

El desarrollo de NexoPostal deja abierta una línea muy interesante de evolución. Lejos de considerarse una aplicación cerrada, la plataforma puede seguir creciendo en varias direcciones, y muchas de ellas no responden a carencias graves, sino a una progresión natural de madurez del producto.

Una primera línea de trabajo futuro consiste en reforzar la robustez de la arquitectura distribuida. Para ello sería recomendable incorporar un sistema de mensajería con garantías de entrega, colas persistentes o patrones outbox/inbox. Esto permite reducir el acoplamiento temporal entre servicios, mejorar la fiabilidad ante caídas parciales o problemas de red y ofrecer mejor trazabilidad de los eventos de negocio.

Una segunda línea estaría orientada a la seguridad y a la operación. Aunque el sistema ya utiliza JWT, refresh tokens y protección básica entre servicios, sería deseable evolucionar hacia autenticación fuerte servicio a servicio mediante JWT internos firmados o mTLS. También podría añadirse auditoría avanzada de acciones sensibles, políticas de rotación de secretos, gestión más formal de certificados y alternativas al correo SMTP tradicional mediante proveedores con API HTTPS.

Una tercera línea se centraría en la experiencia del repartidor. Las mejoras más relevantes en este área serían la navegación embebida en el mapa, el cálculo de ETA por parada, la reordenación inteligente de rutas según tráfico o prioridad y un mejor soporte para segundo plano mediante tecnología nativa o híbrida. Con ello, la app de reparto podría pasar de ser una herramienta web avanzada a una solución de movilidad aún más robusta.

Una cuarta línea de trabajo estaría relacionada con la observabilidad y el control operativo. Sería interesante desplegar dashboards reales con métricas de negocio y de sistema, como volumen de envíos por día, tiempo medio de entrega, entregas fallidas, ocupación por CTA, salud de servicios, latencia de APIs o tasa de errores en sincronización interservicio. Esta información sería valiosa no solo para operación técnica, sino también para toma de decisiones de negocio.

Una quinta línea de evolución tendría un enfoque más funcional y comercial: notificaciones push al cliente, integración con almacenes o ERPs, gestión de reembolsos o seguros, programación de recogidas, soporte para multipaquete y apertura a envíos internacionales. Todas estas opciones amplían el alcance del producto sin romper la base ya construida.

Finalmente, sería muy valioso ampliar la cobertura de pruebas, incorporando más tests de integración y escenarios end to end que recorran los flujos críticos completos: registro, envío, pago, admisión, liberación a reparto, tracking y entrega. Esta línea de mejora tendría un impacto directo en la mantenibilidad futura de la plataforma.

5 CONCLUSIONES

Llevar a cabo el proyecto NexoPostal me ha demostrado que sí es posible diseñar y desarrollar desde cero una plataforma de paquetería completa dentro de los plazos de un TFG. La clave para lograrlo ha estado en mantener una metodología ordenada, acotar muy bien hasta dónde quería llegar y apostar por una arquitectura coherente. Echando la vista atrás, he podido recorrer todas las fases reales del ciclo de vida

del software: desde romperme la cabeza entendiendo el problema y definiendo los requisitos, hasta diseñar la arquitectura, picar el código, integrar los módulos, desplegarlo todo y documentarlo.

Si tuviera que destacar una decisión clave, diría que separar el sistema por dominios (cliente, logística y reparto) utilizando microservicios fue un acierto total. Me ha permitido construir una solución mucho más limpia, fácil de mantener y, sobre todo, realista. No voy a negar que me ha obligado a pelearme con retos complejos de sincronización, seguridad, orquestación y despliegues, pero precisamente eso es lo que creo que le aporta el verdadero valor académico al proyecto.

Por otro lado, incorporar procesos del mundo real ha subido muchísimo el nivel del trabajo. Integrar pagos con Stripe, generar documentos, montar un sistema de tracking en tiempo real con SignalR, gestionar escaneos logísticos o dar soporte offline a la movilidad no son solo "extras". Han hecho que el resultado no sea un conjunto de funcionalidades aisladas, sino un sistema cohesivo donde todas las piezas trabajan juntas para sostener un flujo de negocio real, desde la contratación hasta la entrega en mano.

A nivel personal y formativo, este TFG ha sido un empujón enorme. Me ha servido para consolidar todo lo que he aprendido sobre Angular, ASP.NET Core, Entity Framework, PostgreSQL, Docker, Nginx y GitHub Actions, además de trabajar con APIs externas y seguridad mediante JWT. Pero más allá del stack tecnológico concreto, me llevo algo más importante: he mejorado mi capacidad de análisis, he aprendido a tomar (y defender) decisiones de arquitectura, a organizar mejor mi código y a razonar sobre los requisitos técnicos y de negocio.

Como conclusión final, me quedo con que NexoPostal no solo "funciona", sino que tiene un porqué detrás de cada decisión. Presenta una arquitectura sólida y defendible, flujos de negocio con sentido, un despliegue realista y un margen de mejora que ya tengo identificado. Por todo ello, siento que este proyecto cumple con creces el objetivo del TFG: no es solo una demostración de que sé programar, sino una solución técnicamente consistente, bien estructurada y con capacidad real de seguir creciendo.

6 BIBLIOGRAFÍA

ANGULAR TEAM. ANGULAR DOCUMENTACIÓN. DISPONIBLE EN: [HTTPS://ANGULAR.DEV/](https://angular.dev/)

MICROSOFT. ASP.NET CORE DOCUMENTACIÓN. DISPONIBLE EN:
[HTTPS://LEARN.MICROSOFT.COM/ASPNET/CORE/](https://learn.microsoft.com/aspnet/core/)

MICROSOFT. SIGNALR FOR ASP.NET CORE. DISPONIBLE EN:
[HTTPS://LEARN.MICROSOFT.COM/ASPNET/CORE/SIGNALR/INTRODUCTION](https://learn.microsoft.com/aspnet/core/signalr/introduction)

MICROSOFT. ENTITY FRAMEWORK CORE DOCUMENTACIÓN. DISPONIBLE EN:
[HTTPS://LEARN.MICROSOFT.COM/EF/CORE/](https://learn.microsoft.com/ef/core/)

POSTGRESQL GLOBAL DEVELOPMENT GROUP. POSTGRESQL DOCUMENTATION. DISPONIBLE EN: [HTTPS://WWW.POSTGRESQL.ORG/DOCS/](https://www.postgresql.org/docs/)

DOCKER, INC. DOCKER DOCUMENTATION. DISPONIBLE EN: [HTTPS://DOCS.DOCKER.COM/](https://docs.docker.com/)

NGINX. NGINX DOCUMENTATION. DISPONIBLE EN: [HTTPS://NGINX.ORG/EN/DOCS/](https://nginx.org/en/docs/)

STRIPE, INC. STRIPE CHECKOUT DOCUMENTATION. DISPONIBLE EN:
[HTTPS://DOCS.STRIPE.COM/PAYMENTS/CHECKOUT](https://docs.stripe.com/payments/checkout)

GITHUB. GITHUB ACTIONS DOCUMENTATION. DISPONIBLE EN:
[HTTPS://DOCS.GITHUB.COM/ACTIONS](https://docs.github.com/actions)

LEAFLET CONTRIBUTORS. LEAFLET DOCUMENTATION. DISPONIBLE EN:
[HTTPS://LEAFLETJS.COM/REFERENCE.HTML](https://leafletjs.com/reference.html)

SCANAPP.ORG. HTML5-QR CODE DOCUMENTATION. DISPONIBLE EN:
[HTTPS://SCANAPP.ORG/HTML5-QR CODE-DOCS/](https://scanapp.org/html5-qr-code-docs/)

MDN WEB DOCS. GEOLOCATION API. DISPONIBLE EN: [HTTPS://DEVELOPER.MOZILLA.ORG/](https://developer.mozilla.org/)

7 ANEXOS

...

7.1 ANEXO I. ESTRUCTURA GENERAL DE LA SOLUCIÓN

La solución está formada por los siguientes bloques:

- Frontend clientes.
- Frontend intranet.
- Frontend driver.
- API Gateway.
- Microservicio Auth.
- Microservicio Ciudadano.
- Microservicio Intranet.
- Microservicio Reparto.
- Nginx.
- PostgreSQL por microservicio.
- Integraciones externas: Stripe, SMTP y SignalR.

7.2 ANEXO II. RESUMEN DE ENTIDADES PRINCIPALES

Dominio Auth

- ApplicationUser.

Dominio Ciudadano

- ClientePerfil.
- DireccionFavorita.
- Envio.
- Estados públicos e internos.

Dominio Intranet

- CentroTratamiento.
- OperarioCta.
- OperarioOficina.
- RutaCta.
- AsignacionPaquete.
- MovimientoPaquete.
- Incidencia.

Dominio Reparto

- Repartidor.
- RutaReparto.
- EntregaPaquete.

7.3 ANEXO III. ENDPOINTS RELEVANTES DEL SISTEMA

Auth

- POST /api/auth/login
- POST /api/auth/register
- POST /api/auth/refresh
- GET /api/auth/me
- POST /api/auth/solicitar-reset
- POST /api/auth/reset-password

Ciudadano

- POST /api/envíos/cotizar
- POST /api/envíos/crear
- GET /api/envíos/track/{numero}
- GET /api/envíos/mis-envíos
- GET /api/envíos/etiqueta/{numero}
- GET /api/envíos/factura/{numero}
- GET /api/perfil
- POST /api/perfil
- GET /api/perfil/direcciones
- POST /api/pagos/crear-sesión
- GET /api/pagos/verificar/{sessionId}
- POST /api/pagos/reintentar/{numero}

Intranet

- POST /api/admisión/paquete
- POST /api/admisión/interno/paquete
- POST /api/asignaciones
- PUT /api/asignaciones/{id}/iniciar
- PUT /api/asignaciones/{id}/completar
- POST /api/movimientos
- PUT /api/movimientos/{id}/despachar
- PUT /api/movimientos/{id}/recibir
- POST /api/incidencias

Reparto

- GET /api/reparto/mi-perfil
- GET /api/reparto/ruta
- GET /api/reparto/rutas
- GET /api/reparto/rutas/{id}
- POST /api/reparto/rutas/{id}/iniciar
- POST /api/reparto/rutas/{id}/finalizar
- GET /api/reparto/entregas
- POST /api/reparto/confirmar
- POST /api/reparto/ubicación
- GET /api/reparto/bandeja
- POST /api/reparto/bandeja/{id}/asignar-a-ruta
- POST /api/reparto/interno/bandeja/registrar
- **Anexo V. Posibles diagramas para enriquecer la memoria**
- Diagrama de arquitectura general del sistema.
- Diagrama de despliegue con Docker y Nginx.
- Diagrama de secuencia del flujo cliente -> pago -> admisión -> reparto.
- Diagrama de estados del envío.
- Diagrama entidad-relación simplificado por microservicio.